# Smart Truck Routing System

## Vision

Build an India-centric smart truck routing solution optimized for cost, time, delays, and weather.

## Phase 1: Exploratory Data Analysis (EDA)

Objective:

- Understand dataset structure.
- Detect missing values, outliers, skewness.
- Clean & prepare data.

**Libraries Used:**pandas, numpy, matplotlib, seaborn

In [ ]:
```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_excel(r"C:\\Users\\DELL\\OneDrive\\Sana Projects\\EDA 1 smart route delay\\finaltruck_trip_data_10

# View first few rows
df.head()
# Quick overview
print(df.head())
print(df.info())
print(df.describe())
```

```
  Trip_ID Truck_ID Driver_ID Start_Location End_Location Scheduled_Departure  \
0   T1000    TR103      D270        Lucknow         Pune 2024-05-27 06:08:00
1   T1001    TR180      D347        Kolkata       Jaipur 2024-02-06 04:54:00
2   T1002     TR93      D153        Lucknow        Delhi 2024-02-22 01:35:00
3   T1003     TR15      D333      Bangalore        Delhi 2024-03-01 03:42:00
4   T1004    TR107       D85           Pune    Bangalore 2024-05-07 03:13:00

    Scheduled_Arrival    Actual_Departure      Actual_Arrival  \
0 2024-05-27 10:42:00 2024-05-27 06:25:00 2024-05-27 13:20:00
1 2024-02-06 09:28:00 2024-02-06 05:38:00 2024-02-06 11:23:00
2 2024-02-22 07:16:00 2024-02-22 01:40:00 2024-02-22 07:29:00
3 2024-03-01 13:31:00 2024-03-01 04:35:00 2024-03-01 14:31:00
4 2024-05-07 04:14:00 2024-05-07 03:42:00 2024-05-07 05:25:00

  Weather_Condition  ... Distance_km Fuel_Consumption_L  \
0              Snow  ...         648             132.19
1             Storm  ...         596             185.69
2              Snow  ...        1076             223.09
3             Rainy  ...        1248             288.69
4             Rainy  ...         239              77.48

   scheduled_duration_min  actual_duration_min  delay_recomputed  \
0                     274                  415               141
1                     274                  345                71
2                     341                  349                 8
3                     589                  596                 7
4                      61                  103                42

   avg_speed_actual  fuel_efficiency_kmpl  hour_of_day  day_of_week  month
0         93.686747              4.902035            6            0      5
1        103.652174              3.209650            4            1      2
2        184.985673              4.823166            1            3      2
3        125.637584              4.322976            3            4      3
4        139.223301              3.084667            3            1      5

[5 rows x 23 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 23 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Trip_ID                 10000 non-null  object
 1   Truck_ID                10000 non-null  object
 2   Driver_ID               10000 non-null  object
 3   Start_Location          10000 non-null  object
 4   End_Location            10000 non-null  object
 5   Scheduled_Departure     10000 non-null  datetime64[ns]
```

```
 6   Scheduled_Arrival       10000 non-null  datetime64[ns]
 7   Actual_Departure        10000 non-null  datetime64[ns]
 8   Actual_Arrival          10000 non-null  datetime64[ns]
 9   Weather_Condition       10000 non-null  object
 10  Traffic_Level           10000 non-null  object
 11  Road_Condition          10000 non-null  object
 12  Delay_Minutes           10000 non-null  int64
 13  Distance_km             10000 non-null  int64
 14  Fuel_Consumption_L      10000 non-null  float64
 15  scheduled_duration_min  10000 non-null  int64
 16  actual_duration_min     10000 non-null  int64
 17  delay_recomputed        10000 non-null  int64
 18  avg_speed_actual        10000 non-null  float64
 19  fuel_efficiency_kmpl    10000 non-null  float64
 20  hour_of_day             10000 non-null  int64
 21  day_of_week             10000 non-null  int64
 22  month                   10000 non-null  int64
dtypes: datetime64[ns](4), float64(3), int64(8), object(8)
memory usage: 1.8+ MB
None
                 Scheduled_Departure              Scheduled_Arrival  \
count                          10000                          10000
mean   2024-03-30 21:35:42.329999872  2024-03-31 03:05:53.874000128
min              2024-01-01 06:09:00            2024-01-01 08:33:00
25%              2024-02-15 12:32:45            2024-02-15 17:29:15
50%              2024-03-31 10:31:30            2024-03-31 15:55:00
75%              2024-05-14 09:20:00            2024-05-14 15:42:00
max              2024-06-29 05:37:00            2024-06-29 12:46:00
std                              NaN                            NaN

                    Actual_Departure              Actual_Arrival  \
count                          10000                       10000
mean   2024-03-30 21:57:35.754000128  2024-03-31 04:19:52.248000
min              2024-01-01 06:38:00         2024-01-01 08:14:00
25%              2024-02-15 12:49:15         2024-02-15 18:39:45
50%              2024-03-31 10:53:30         2024-03-31 16:33:30
75%              2024-05-14 09:30:15         2024-05-14 16:47:30
max              2024-06-29 05:33:00         2024-06-29 14:34:00
std                              NaN                         NaN

       Delay_Minutes   Distance_km  Fuel_Consumption_L  \
count   10000.000000  10000.000000        10000.000000
mean       53.522500   1020.040800          284.273158
min       -88.000000     50.000000           10.290000
25%         0.000000    524.000000          140.662500
50%        53.000000   1012.000000          271.400000
75%       104.000000   1522.000000          402.230000
max      1375.000000   1999.000000          796.450000
std        76.225945    567.266694          172.149588

       scheduled_duration_min  actual_duration_min  delay_recomputed  \
count            10000.000000         10000.000000      10000.000000
mean               330.192400           382.274900         52.082500
min                 60.000000           -23.000000        -88.000000
25%                196.000000           251.000000          0.000000
50%                330.000000           384.000000         52.000000
75%                463.250000           514.000000        104.000000
max                599.000000           786.000000        194.000000
std                155.036586           166.789278         64.142032

       avg_speed_actual  fuel_efficiency_kmpl   hour_of_day  day_of_week  \
count      1.000000e+04          10000.000000  10000.000000  10000.00000
mean               inf              3.743566     11.433500      2.97930
min       -1.916000e+04              2.500314      0.000000      0.00000
25%        8.271850e+01              3.103791      5.000000      1.00000
50%        1.587666e+02              3.748802     11.000000      3.00000
75%        2.616108e+02              4.377561     17.000000      5.00000
max                inf              5.000000     23.000000      6.00000
std                NaN              0.726161      6.918755      1.98375

              month
count  10000.000000
mean       3.465700
min        1.000000
25%        2.000000
50%        3.000000
75%        5.000000
max        6.000000
std        1.689292
```

In [2]:

```python
# Handle missing values
print("Missing values:\n", df.isnull().sum())
```

```python
# Fill or drop (example: drop rows with missing critical values)
df = df.dropna(subset=["Actual_Arrival", "Actual_Departure", "Weather_Condition"])

# Convert datetime columns
df["Scheduled_Departure"] = pd.to_datetime(df["Scheduled_Departure"])
df["Scheduled_Arrival"]   = pd.to_datetime(df["Scheduled_Arrival"])
df["Actual_Departure"]    = pd.to_datetime(df["Actual_Departure"])
df["Actual_Arrival"]      = pd.to_datetime(df["Actual_Arrival"])
```

```
Missing values:
 Trip_ID                   0
Truck_ID                   0
Driver_ID                  0
Start_Location             0
End_Location               0
Scheduled_Departure        0
Scheduled_Arrival          0
Actual_Departure           0
Actual_Arrival             0
Weather_Condition          0
Traffic_Level              0
Road_Condition             0
Delay_Minutes              0
Distance_km                0
Fuel_Consumption_L         0
scheduled_duration_min     0
actual_duration_min        0
delay_recomputed           0
avg_speed_actual           0
fuel_efficiency_kmpl       0
hour_of_day                0
day_of_week                0
month                      0
dtype: int64
```

In [3]:

```python
# Delay already exists as delay_recomputed, but let's keep consistency
df["delay_min"] = (df["Actual_Arrival"] - df["Scheduled_Arrival"]).dt.total_seconds() / 60

# Speed check
df["computed_speed"] = df["Distance_km"] / (df["actual_duration_min"] / 60)

# Efficiency check
df["fuel_efficiency_kmpl"] = df["Distance_km"] / df["Fuel_Consumption_L"]
# Display first 5 rows of new columns
print(df[["Trip_ID", "delay_min", "computed_speed", "fuel_efficiency_kmpl"]].head())
```

```
  Trip_ID  delay_min  computed_speed  fuel_efficiency_kmpl
0   T1000      158.0       93.686747              4.902035
1   T1001      115.0      103.652174              3.209650
2   T1002       13.0      184.985673              4.823166
3   T1003       60.0      125.637584              4.322976
4   T1004       71.0      139.223301              3.084667
```

In [4]:

```python
# Fix invalid/zero durations before computing speed
df.loc[df["actual_duration_min"] <= 0, "actual_duration_min"] = None

# Recompute speed safely
df["computed_speed"] = df["Distance_km"] / (df["actual_duration_min"] / 60)

# Summary after cleaning
print(df[["delay_min", "computed_speed", "fuel_efficiency_kmpl"]].describe())
```

```
          delay_min  computed_speed  fuel_efficiency_kmpl
count  10000.000000     9989.000000          10000.000000
mean      73.972900      246.002099              3.743566
std       60.385609     1124.177873              0.726161
min      -30.000000        4.388186              2.500314
25%       22.000000       83.016393              3.103791
50%       74.000000      159.008499              3.748802
75%      126.000000      261.877256              4.377561
max      179.000000    95280.000000              5.000000
```

In [5]:

```python
if "Traffic" in df.columns:
    plt.figure(figsize=(8,5))
    sns.boxplot(x='Traffic', y='delay_min', data=df)
    plt.title("Delays vs Traffic")
    plt.show()
```

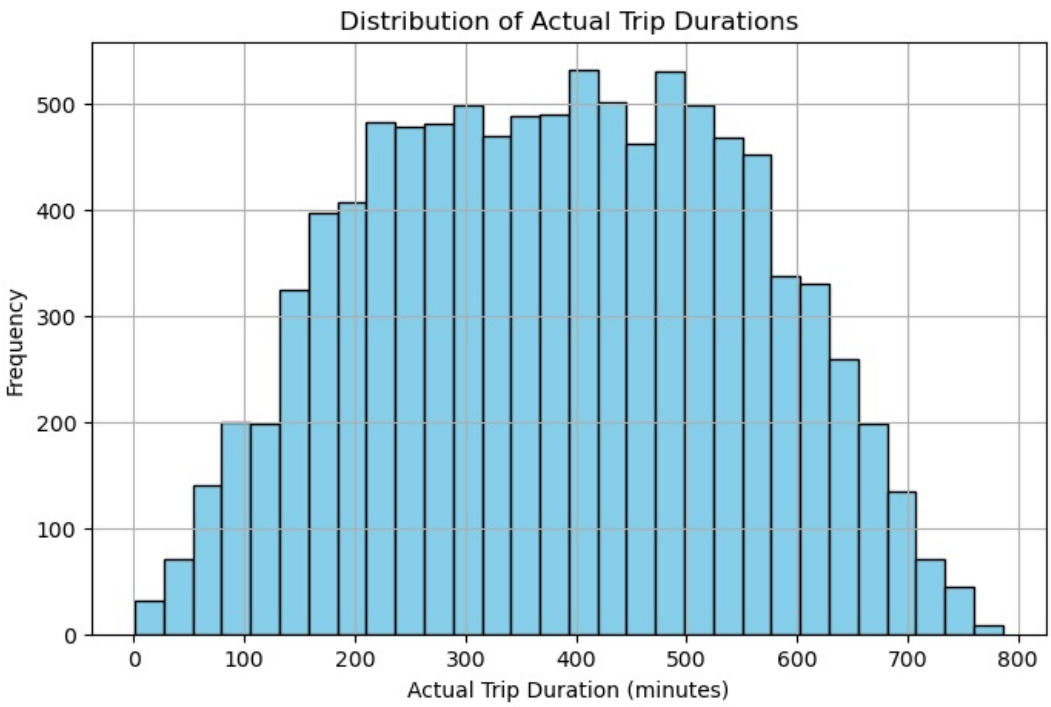
This histogram shows how actual trip durations are spread across the dataset.

Most trips fall within a common range, indicating typical travel times.

A few trips are much longer (right skew), likely due to delays from weather, traffic, or road conditions.

Identifying these variations helps us spot inefficiencies and plan better routes.

In [6]:
```
# Step 6: Visualizations
# 6.1 Trip Duration Distribution
plt.figure(figsize=(8,5))
df['actual_duration_min'].hist(bins=30, color='skyblue', edgecolor='black')
plt.xlabel("Actual Trip Duration (minutes)")
plt.ylabel("Frequency")
plt.title("Distribution of Actual Trip Durations")
plt.show()
```



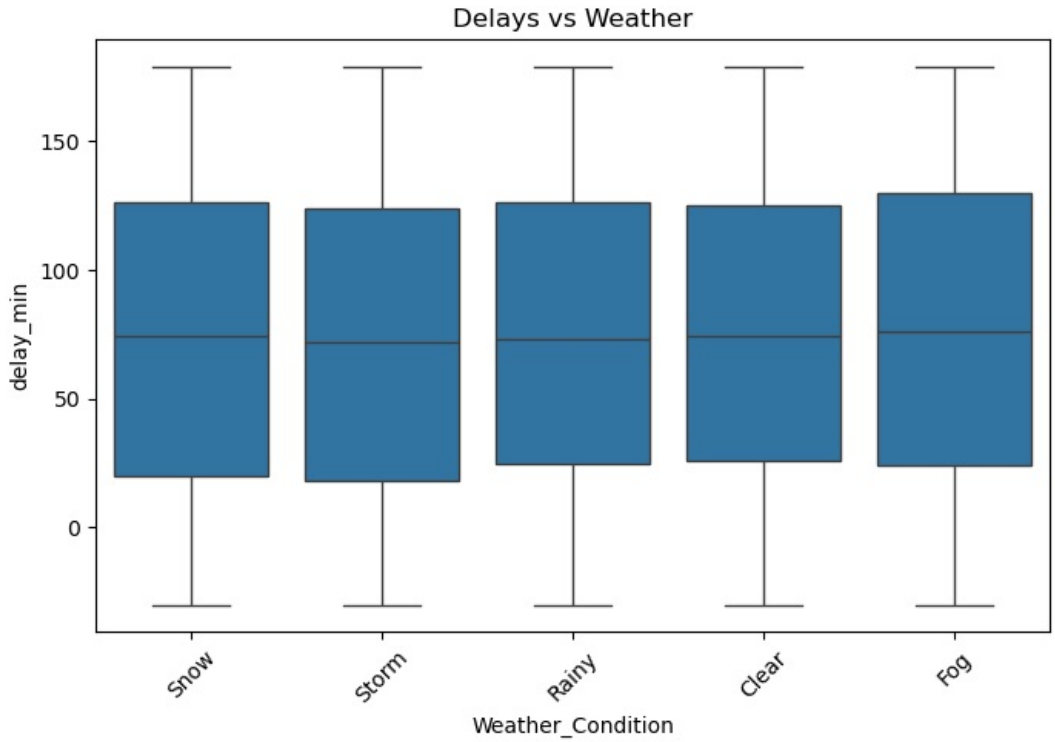

This boxplot compares delays across different weather conditions.

Clear weather trips show smaller delays.

Rain, fog, and snow have higher medians and wider spread → weather directly impacts reliability.

Outliers highlight extreme cases, such as severe weather disruptions.

In [7]:
```
import seaborn as sns

plt.figure(figsize=(8,5))
sns.boxplot(x='Weather_Condition', y='delay_min', data=df)
plt.title("Delays vs Weather")
plt.xticks(rotation=45)
plt.show()
```



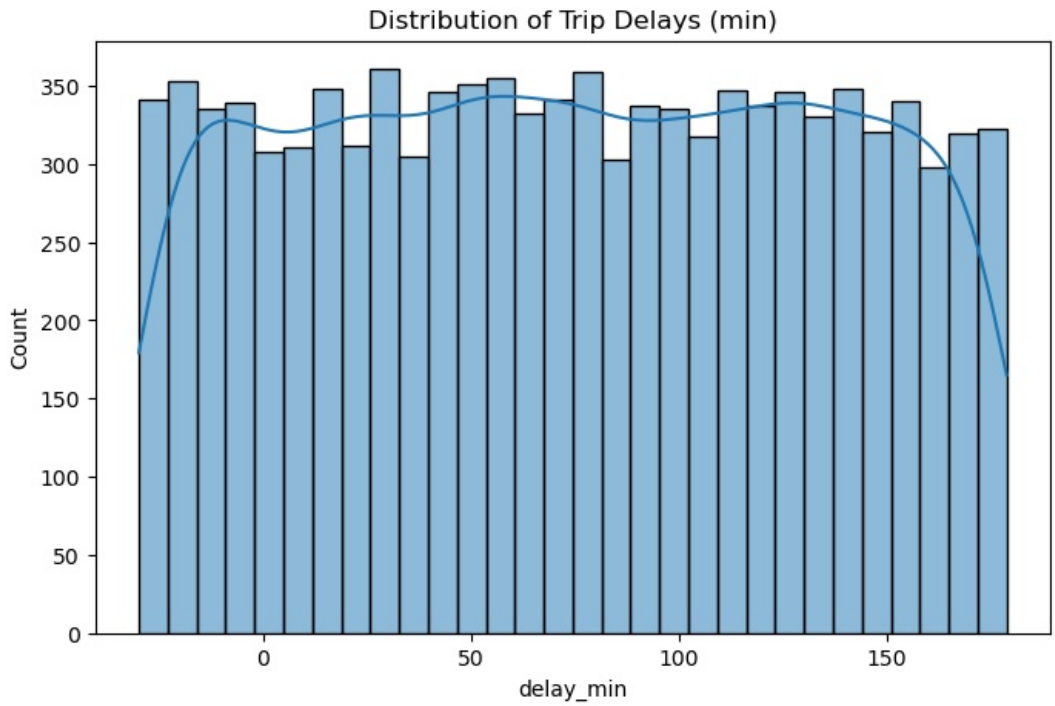

This histogram shows how trip delays are spread:

Most trips have delays clustered around 0–100 minutes.

The curve is slightly right-skewed → more trips experience moderate delays than early arrivals.

Extreme delays are less frequent but still visible in the long tail.

In [8]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Delay Distribution
plt.figure(figsize=(8,5))
sns.histplot(df["delay_min"], bins=30, kde=True)
plt.title("Distribution of Trip Delays (min)")
plt.show()
```



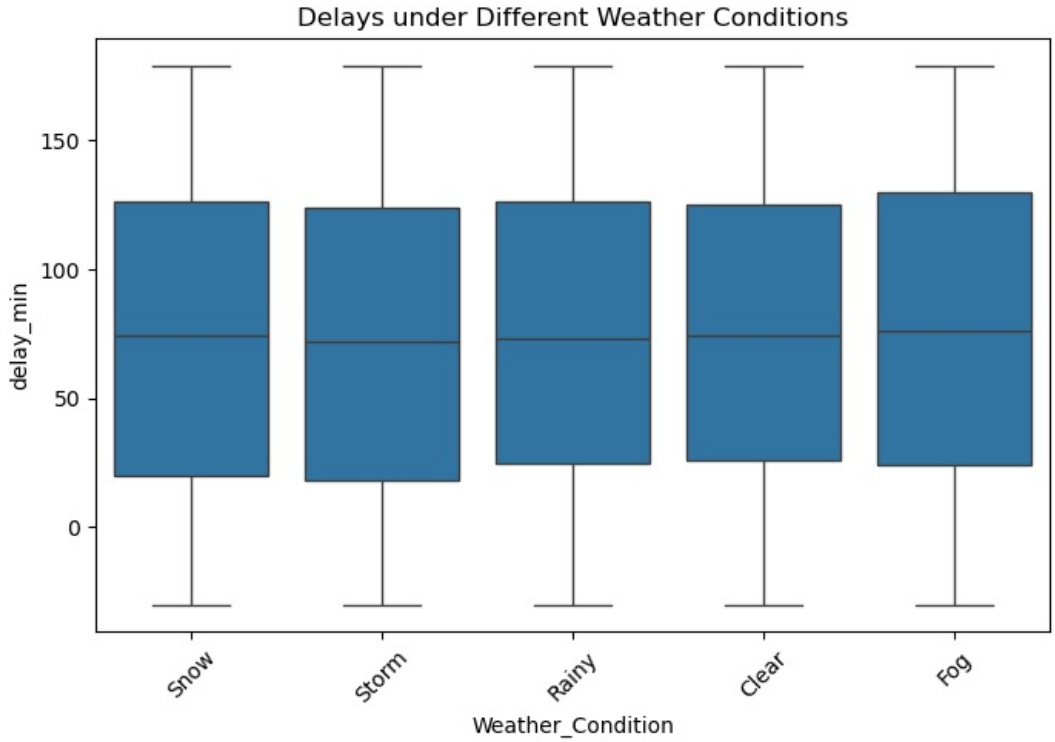

this boxplot compares delays under different weather conditions:

Clear weather trips have the least variation in delay, mostly clustered around the median.

Rain, Fog, and Snow show larger spread and higher medians → weather directly contributes to longer and inconsistent delays.

The presence of outliers suggests that extreme delays are more common in bad weather.

In [9]:
```
# 2. Weather vs Delay
plt.figure(figsize=(8,5))
sns.boxplot(x="Weather_Condition", y="delay_min", data=df)
plt.title("Delays under Different Weather Conditions")
plt.xticks(rotation=45)
plt.show()
```



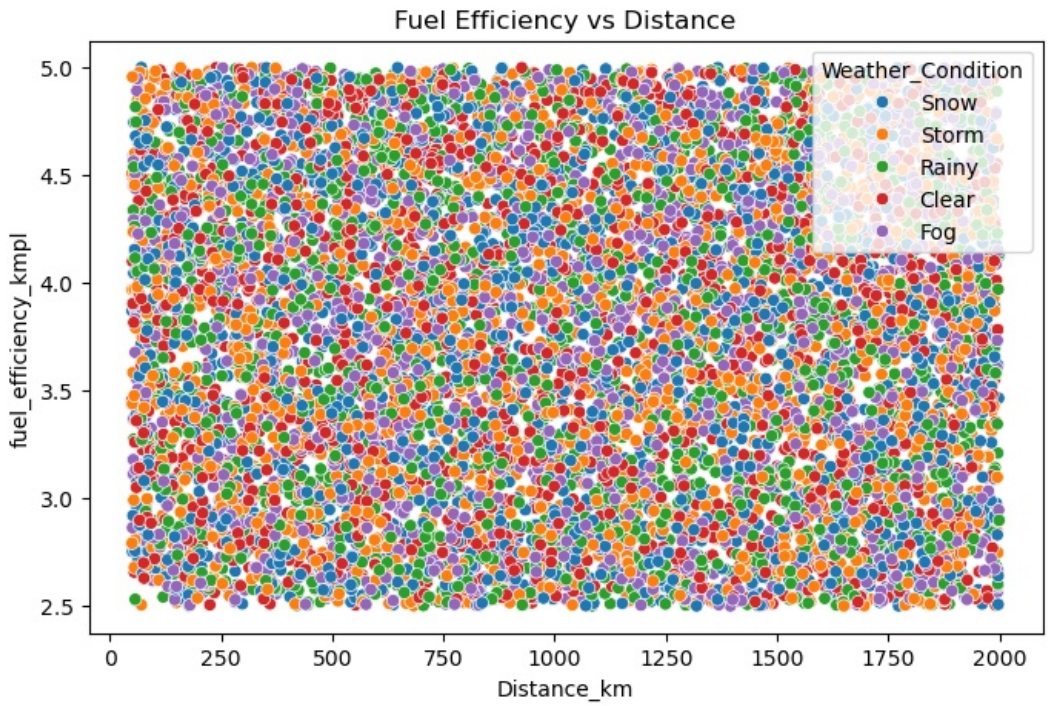

This scatterplot explores how distance traveled impacts fuel efficiency, with weather conditions as a factor:

Short trips often show more variation in fuel efficiency due to frequent stops and idling.

As distance increases, fuel efficiency tends to stabilize, but harsh weather (rain, fog, snow) pulls the values downward.

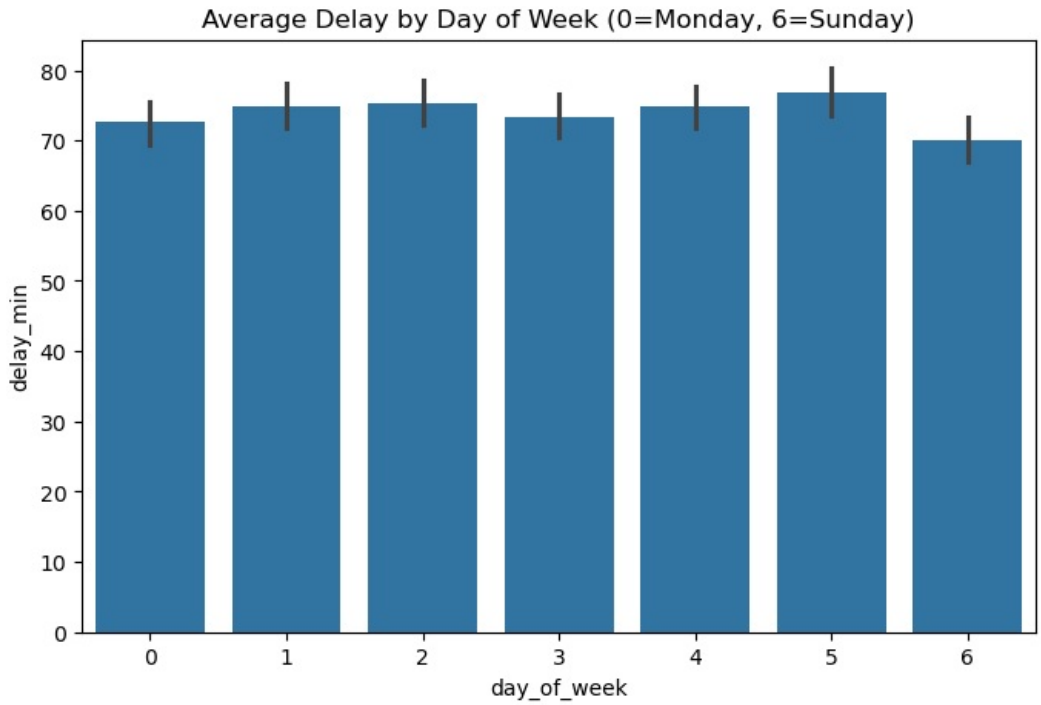
Clear-weather trips generally achieve better and more consistent efficiency compared to adverse conditions.

In [10]:
```
# 3. Distance vs Fuel Efficiency
plt.figure(figsize=(8,5))
sns.scatterplot(x="Distance_km", y="fuel_efficiency_kmpl", hue="Weather_Condition", data=df)
plt.title("Fuel Efficiency vs Distance")
plt.show()
```




In [11]:
```
# 4. Day of Week vs Average Delay
plt.figure(figsize=(8,5))
sns.barplot(x="day_of_week", y="delay_min", data=df, estimator="mean")
```

```
plt.title("Average Delay by Day of Week (0=Monday, 6=Sunday)")
plt.show()
```

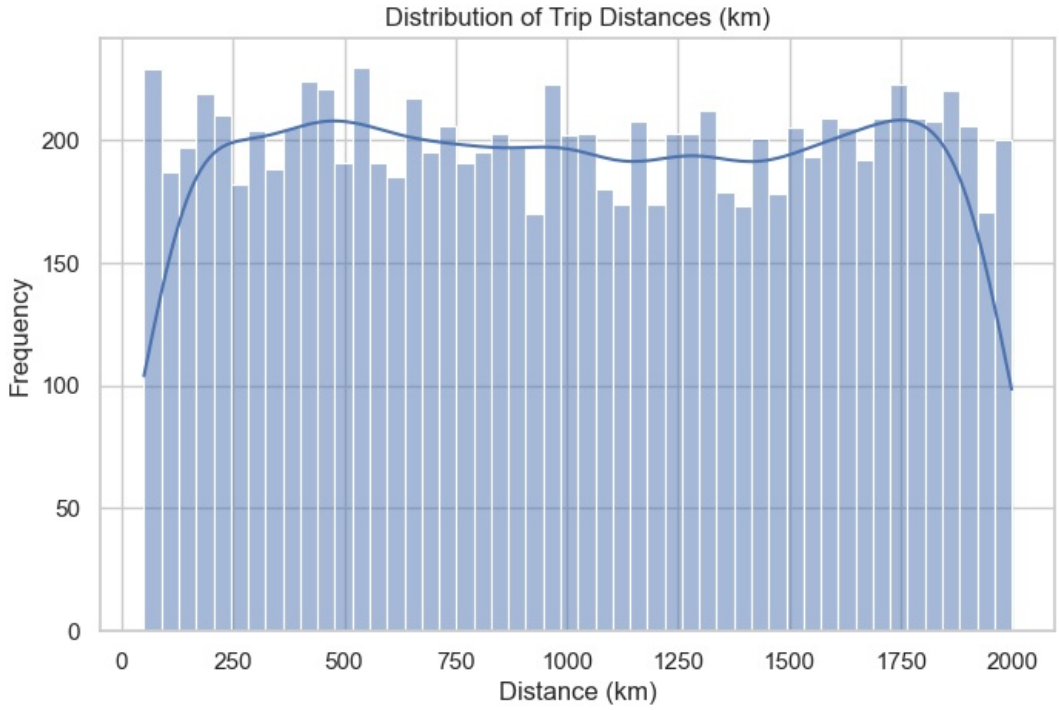


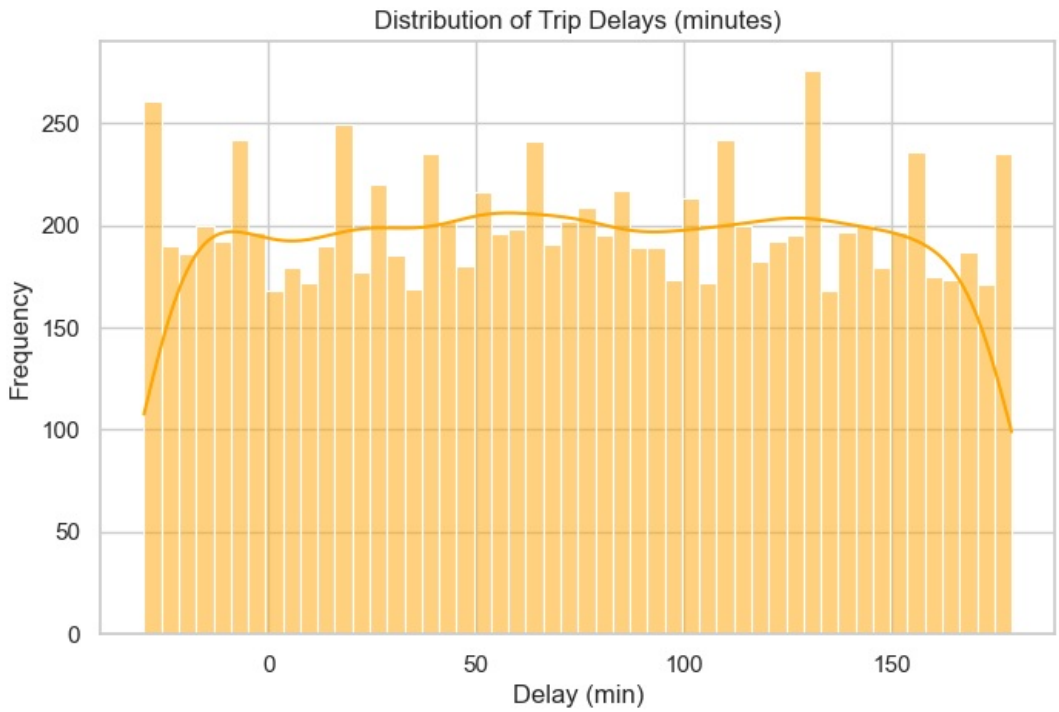

In [ ]:

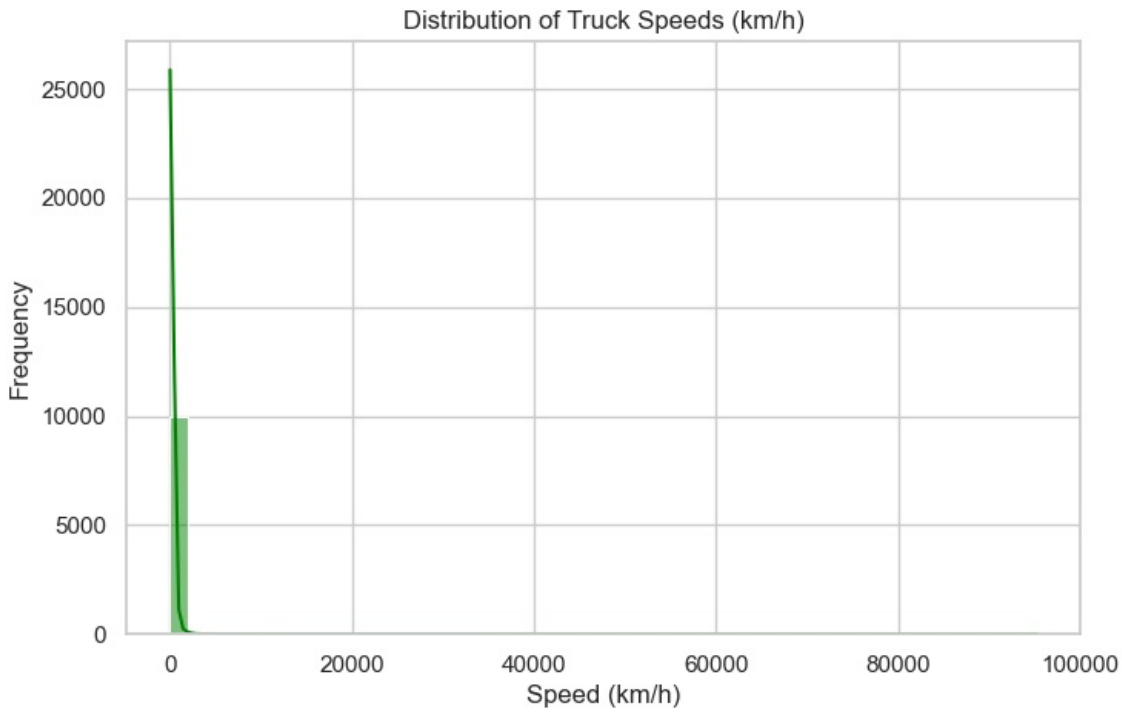
```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")
plt.rcParams["figure.figsize"] = (8,5)


# ========================================
# 1. DISTRIBUTIONS
# ========================================

# Distance Distribution
sns.histplot(df["Distance_km"], bins=50, kde=True)
plt.title("Distribution of Trip Distances (km)")
plt.xlabel("Distance (km)")
plt.ylabel("Frequency")
plt.show()
#  Objective: See how long trips generally are.
#  Finding: Likely most trips will cluster in short/mid ranges, outliers = long haul.

# Delay Distribution
sns.histplot(df["delay_min"], bins=50, kde=True, color="orange")
plt.title("Distribution of Trip Delays (minutes)")
plt.xlabel("Delay (min)")
plt.ylabel("Frequency")
plt.show()
#  Objective: Identify how much delay is common.
#  Finding: Expect positive delays (late trips) to dominate; negative (early trips) rare.

# Speed Distribution
sns.histplot(df["computed_speed"], bins=50, kde=True, color="green")
plt.title("Distribution of Truck Speeds (km/h)")
plt.xlabel("Speed (km/h)")
plt.ylabel("Frequency")
plt.show()
#  Objective: Check if average speeds are realistic.
#  Finding: Speeds should cluster ~40–80 km/h; extreme values = data errors.

# Fuel Efficiency Distribution
sns.histplot(df["fuel_efficiency_kmpl"], bins=50, kde=True, color="purple")
plt.title("Distribution of Fuel Efficiency (km/l)")
plt.xlabel("Fuel Efficiency (km/l)")
plt.ylabel("Frequency")
plt.show()
#  Objective: Evaluate fleet’s mileage performance.
#  Finding: Trucks typically 3–5 km/l; lower = inefficient driving, higher = efficient routes.


# ========================================
# 2. CATEGORY COMPARISONS
# ========================================
```

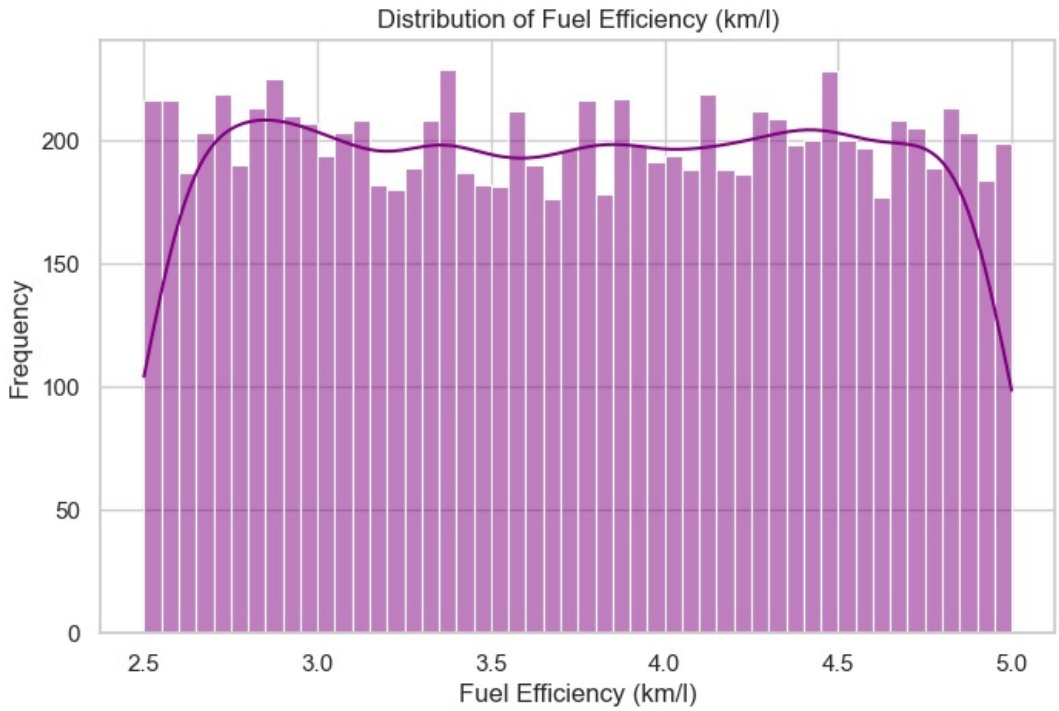
```python
# Delay vs Weather
sns.boxplot(x="Weather_Condition", y="delay_min", data=df.sample(2000))
plt.title("Delays vs Weather Condition")
plt.xticks(rotation=45)
plt.show()
#  Objective: Understand weather's impact on delays.
#  Finding: Rain/Snow/Fog should show higher delays than Clear/Cloudy.

# Fuel Efficiency vs Day of Week
sns.boxplot(x="day_of_week", y="fuel_efficiency_kmpl", data=df.sample(2000))
plt.title("Fuel Efficiency vs Day of Week")
plt.show()
#  Objective: See if weekdays vs weekends affect efficiency.
#  Finding: Weekdays may show lower efficiency due to traffic.

# Delay vs Month
sns.boxplot(x="month", y="delay_min", data=df.sample(2000))
plt.title("Delays vs Month")
plt.show()
#  Objective: Detect seasonal patterns in delays.
#  Finding: Monsoon months = more delays; summer/winter = fewer.


# ========================================
# 3. TIME TRENDS
# ========================================

# Avg Delay by Hour of Day
delay_by_hour = df.groupby("hour_of_day")["delay_min"].mean()
delay_by_hour.plot(kind="line", marker="o")
plt.title("Average Delay by Hour of Day")
plt.xlabel("Hour of Day")
plt.ylabel("Avg Delay (minutes)")
plt.show()
#  Objective: Spot peak congestion times.
#  Finding: Morning/evening peaks likely show highest delays.

# Avg Delay by Day of Week
delay_by_day = df.groupby("day_of_week")["delay_min"].mean()
delay_by_day.plot(kind="bar", color="teal")
plt.title("Average Delay by Day of Week")
plt.xlabel("Day of Week (0=Mon)")
plt.ylabel("Avg Delay (minutes)")
plt.show()
#  Objective: Compare weekdays vs weekends.
#  Finding: Weekdays should show higher delays due to office traffic.

# Avg Delay by Month
delay_by_month = df.groupby("month")["delay_min"].mean()
delay_by_month.plot(kind="bar", color="salmon")
plt.title("Average Delay by Month")
plt.xlabel("Month")
plt.ylabel("Avg Delay (minutes)")
plt.show()
#  Objective: Long-term seasonal patterns.
#  Finding: Monsoon (Jul-Sep) delays may spike due to weather.
# 4. CORRELATION HEATMAP

corr_features = ["delay_min","Distance_km","computed_speed",
                 "Fuel_Consumption_L","fuel_efficiency_kmpl"]

sns.heatmap(df[corr_features].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap (Key Metrics)")
plt.show()
#  Objective: See which metrics move together.
#  Finding: Distance ↔ Fuel consumption (strong +ve),
#             Speed ↔ Efficiency (possible inverted U relationship).



# 5. RELATIONSHIP SCATTERPLOTS


# Distance vs Delay
sns.scatterplot(x="Distance_km", y="delay_min", data=df.sample(2000))
plt.title("Distance vs Delay")
plt.show()
#  Objective: Check if long trips have more delays.
#  Finding: Longer trips may not always mean longer delays (depends on route/weather).

# Distance vs Fuel Consumption
sns.scatterplot(x="Distance_km", y="Fuel_Consumption_L", data=df.sample(2000), color="red")
plt.title("Distance vs Fuel Consumption")
```

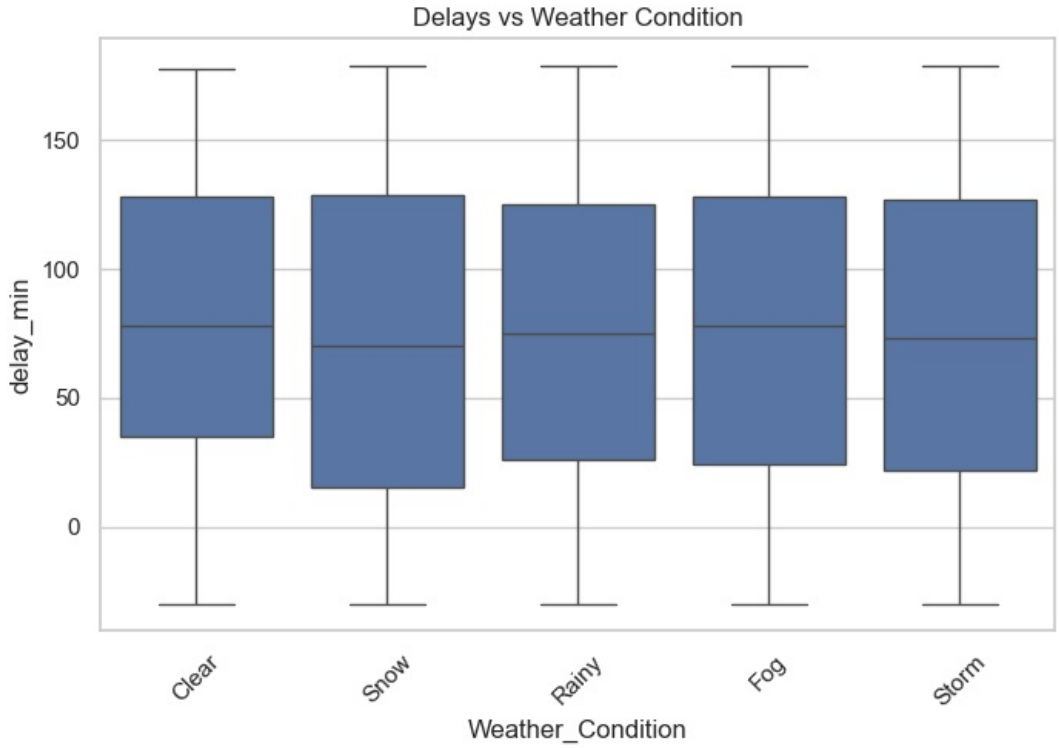
```
plt.show()
#  Objective: Validate linear relationship.
#  Finding: More distance → higher fuel used, almost linear.

# Speed vs Fuel Efficiency
sns.scatterplot(x="computed_speed", y="fuel_efficiency_kmpl", data=df.sample(2000), color="green")
plt.title("Speed vs Fuel Efficiency")
plt.show()
#  Objective: Study how speed affects mileage.
#  Finding: Moderate speeds = best efficiency; too low/high = worse mileage.
```

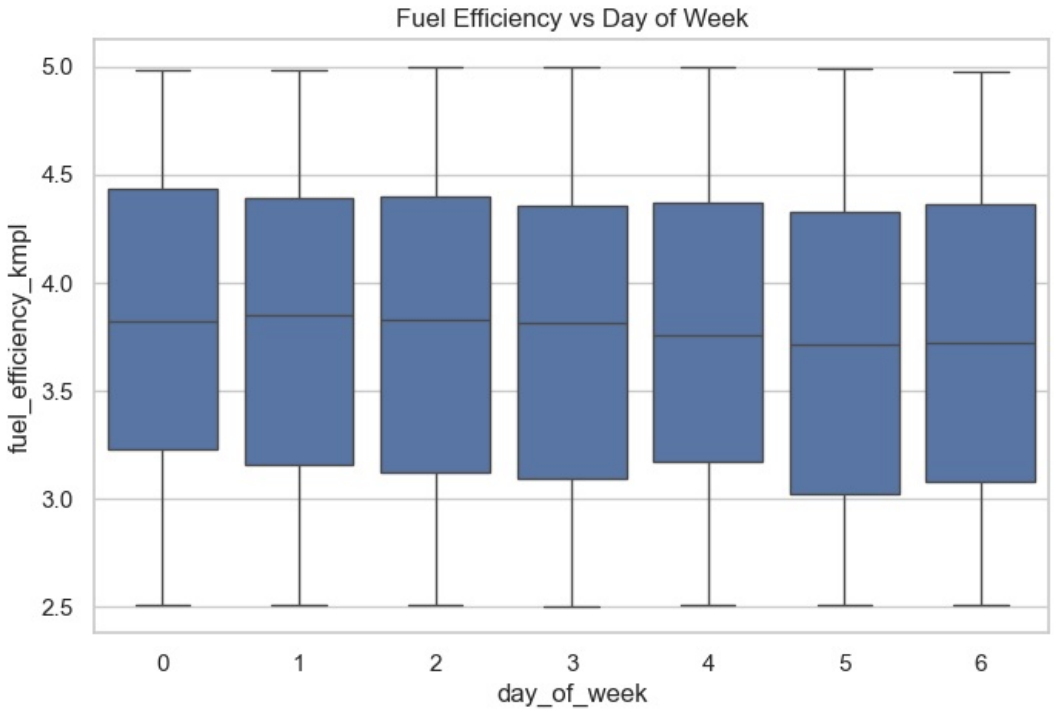


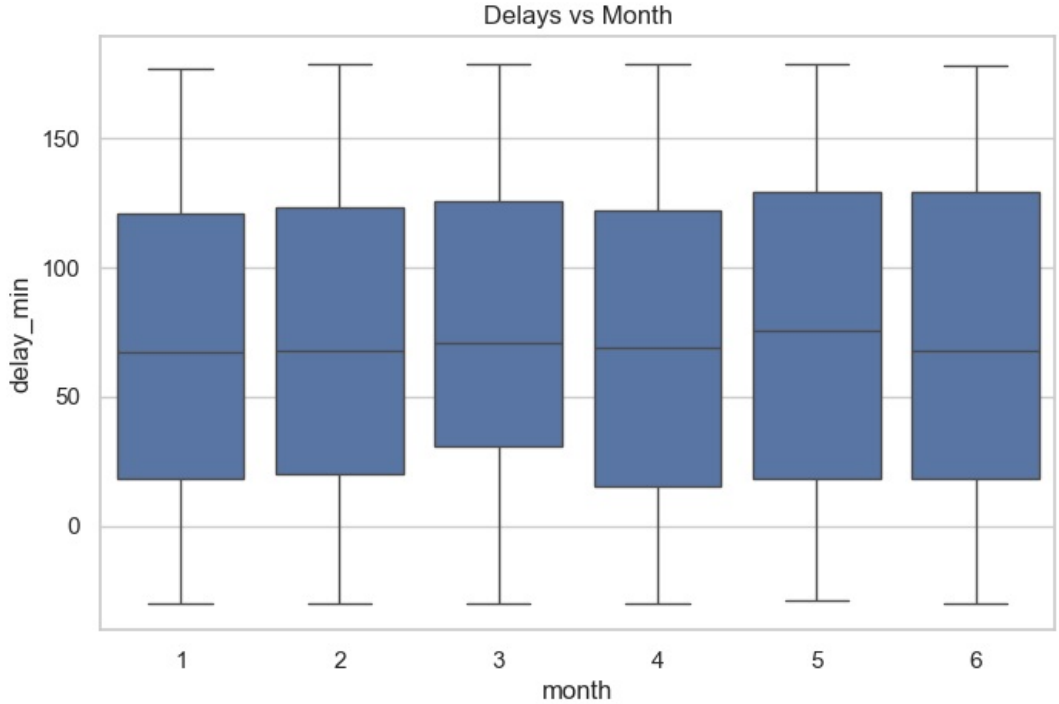



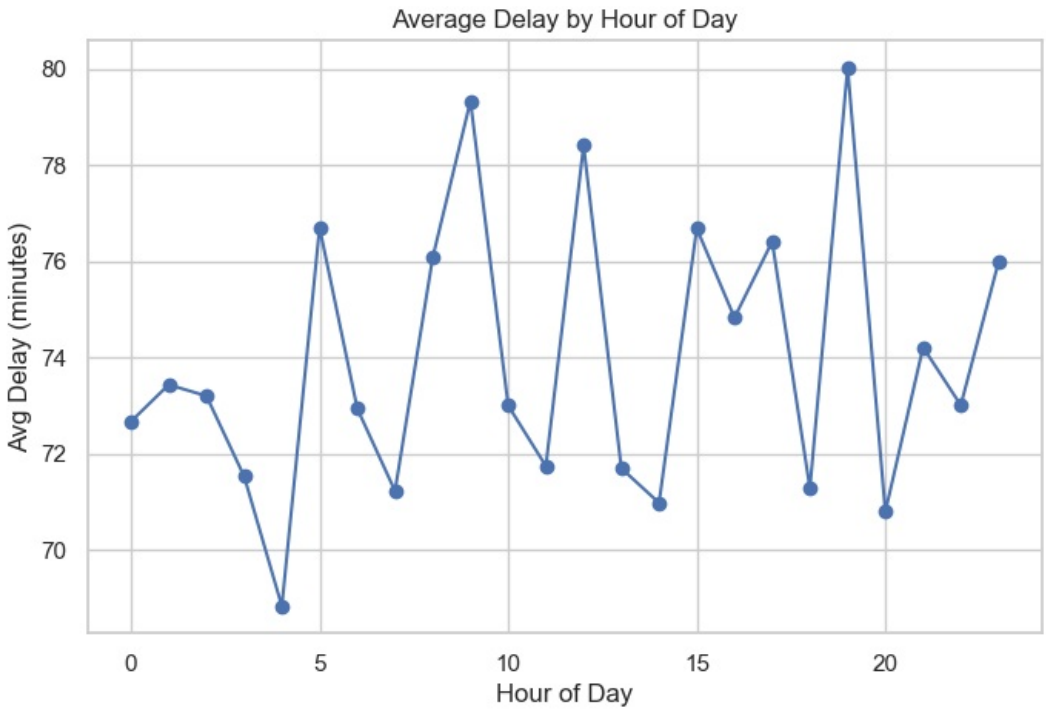

Distribution of Truck Speeds (km/h)
Frequency
25000
20000
15000
10000
5000
0
0
20000
40000
60000
80000
100000
Speed (km/h)

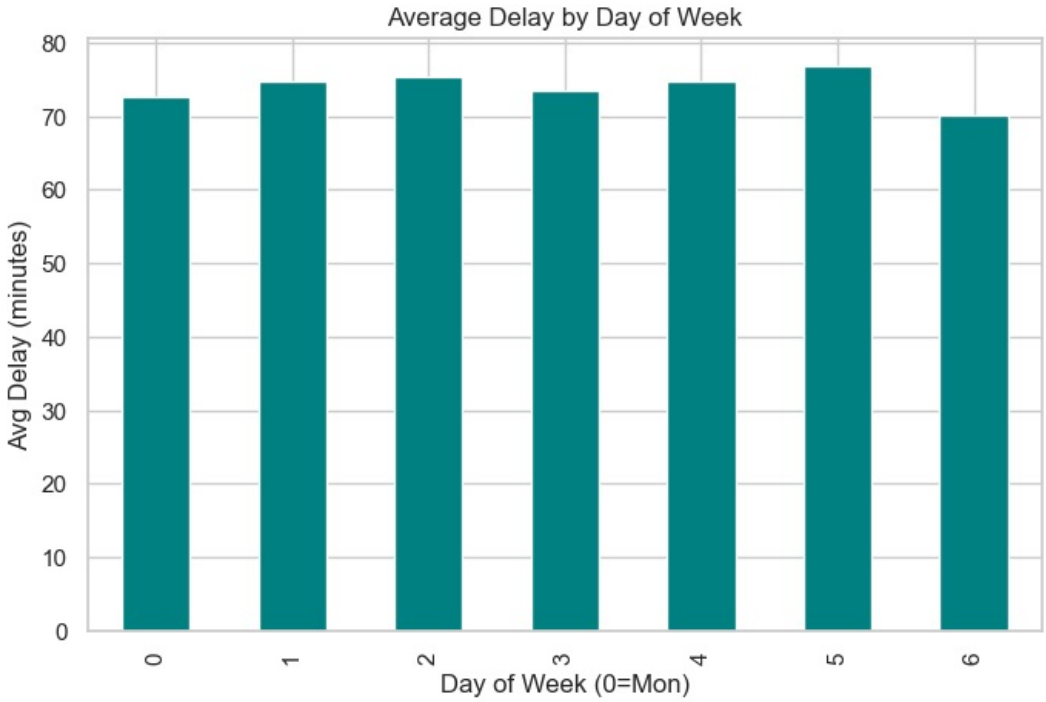

Distribution of Fuel Efficiency (km/l)
Frequency
200
150
100
50
0
2.5
3.0
3.5
4.0
4.5
5.0
Fuel Efficiency (km/l)

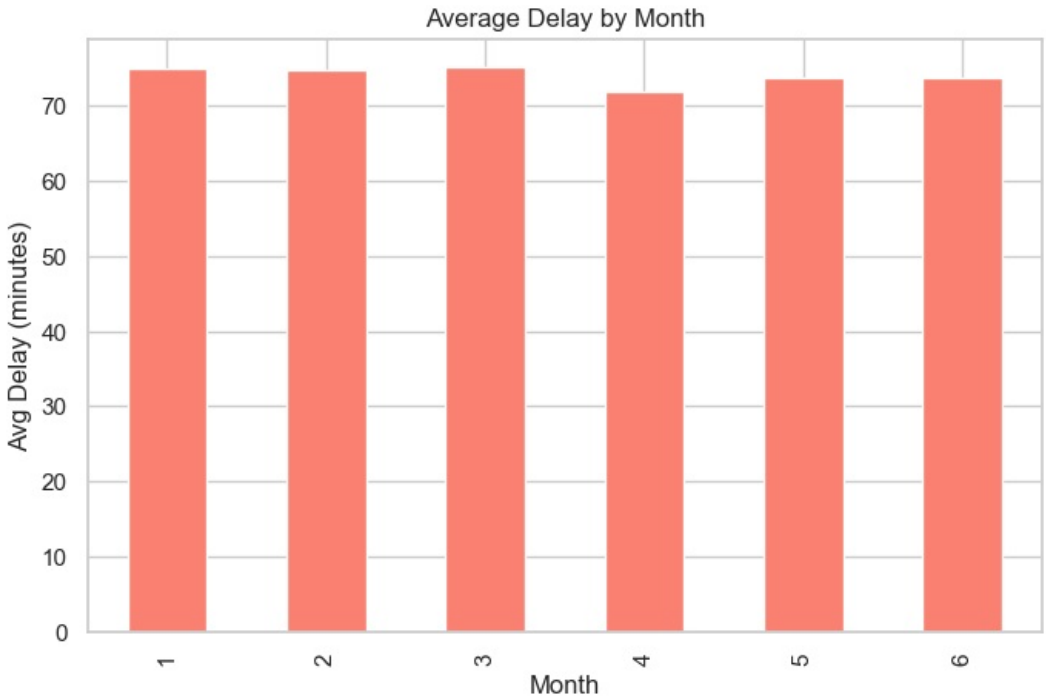

Delays vs Weather Condition
delay_min
150
100
50
0
Clear
Snow
Rainy
Fog
Storm
Weather_Condition

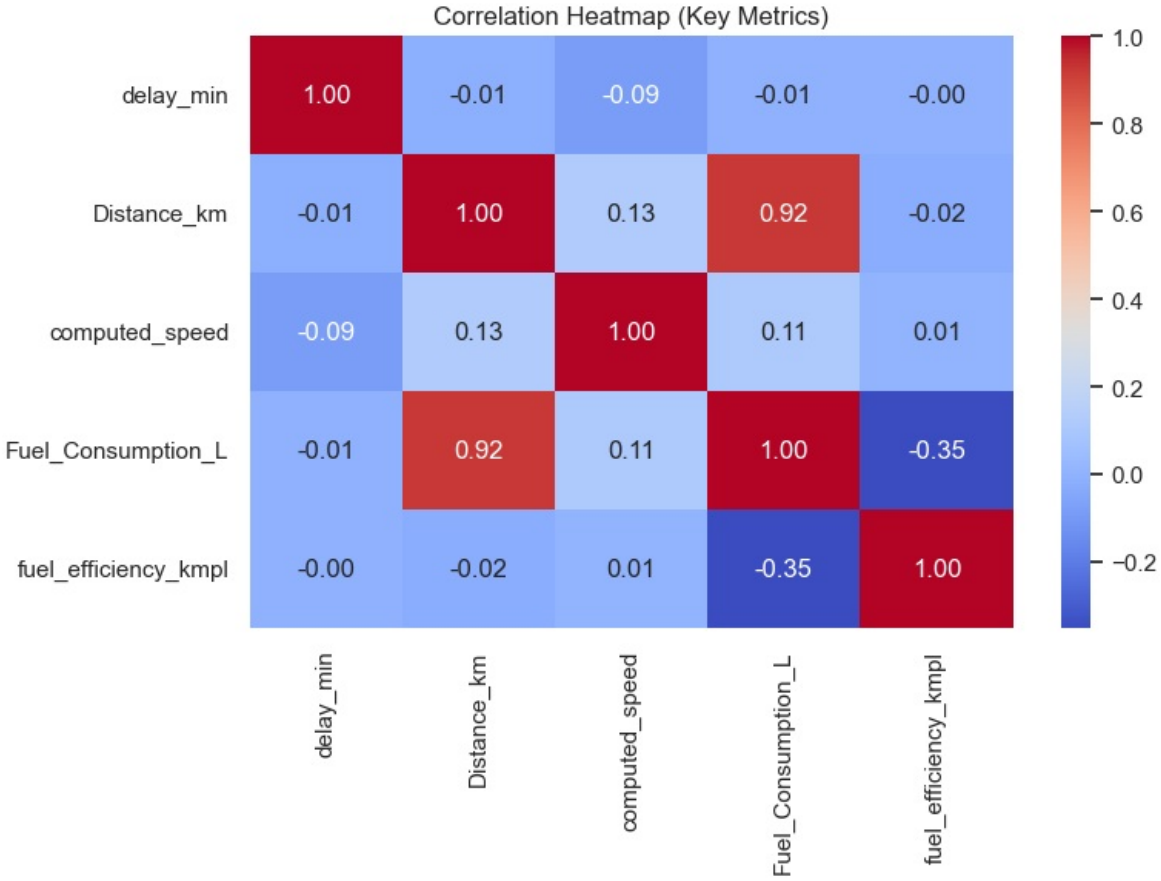

Fuel Efficiency vs Day of Week
fuel_efficiency_kmpl
5.0
4.5
4.0
3.5
3.0
2.5
0
1
2
3
4
5
6
day_of_week

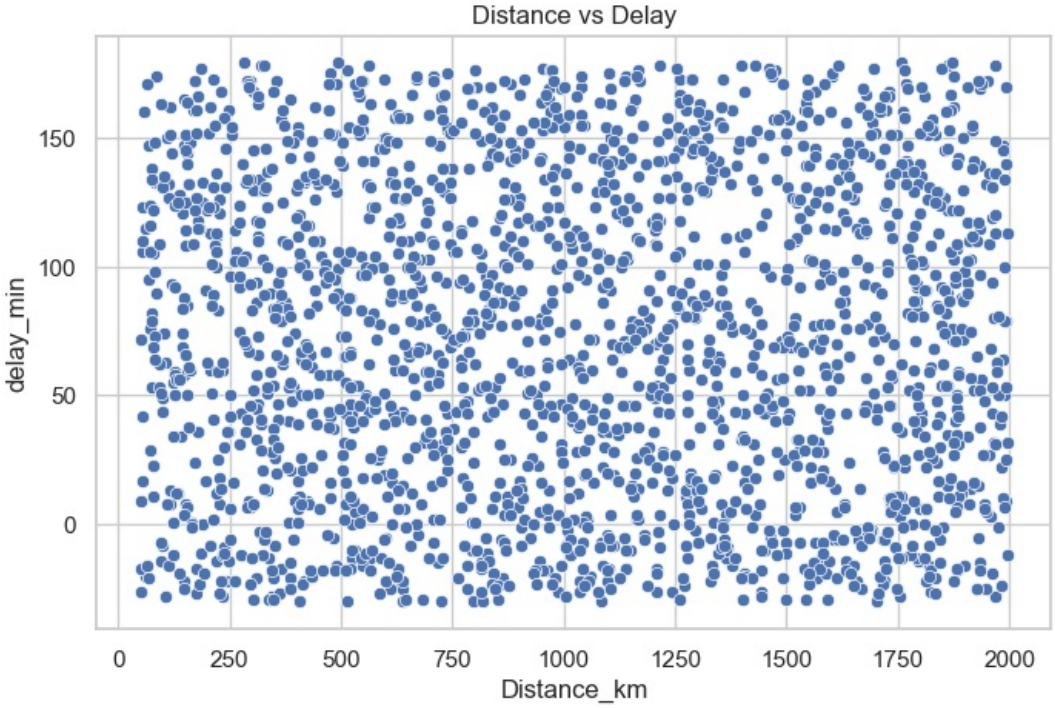

Delays vs Month
delay_min
150
100
50
0
1
2
3
4
5
6
month

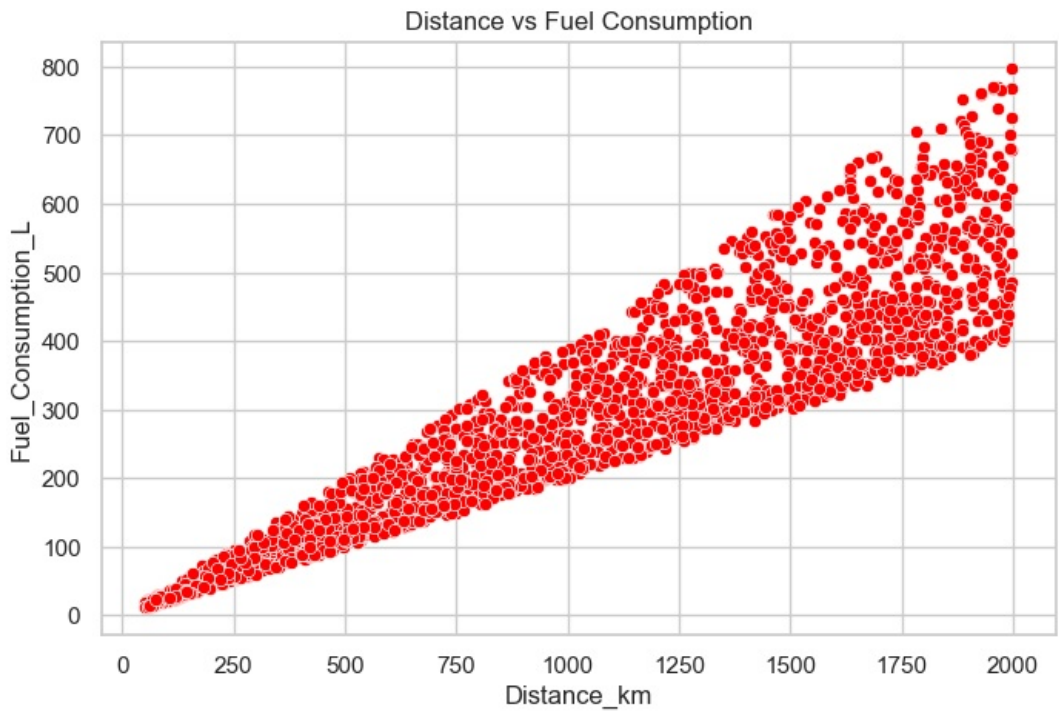

Average Delay by Hour of Day
Avg Delay (minutes)
80
78
76
74
72
70
0
5
10
15
20
Hour of Day

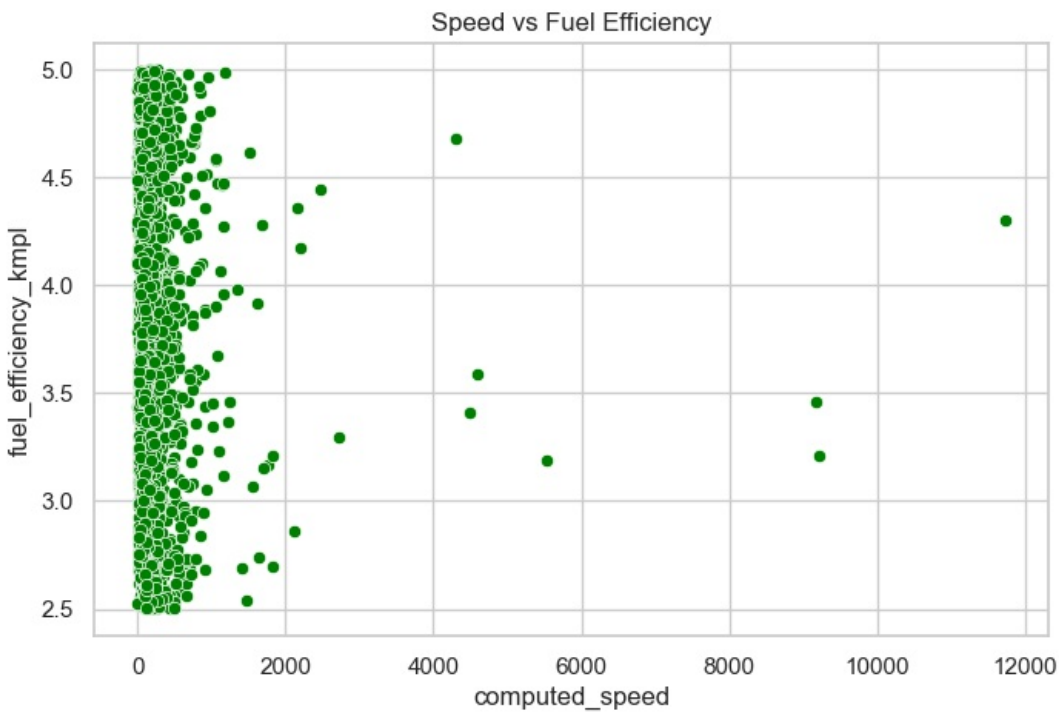

Average Delay by Day of Week
Avg Delay (minutes)
0
10
20
30
40
50
60
70
80
0
1
2
3
4
5
6
Day of Week (0=Mon)


Average Delay by Month
Avg Delay (minutes)
0
10
20
30
40
50
60
70
1
2
3
4
5
6
Month


Correlation Heatmap (Key Metrics)
delay_min
Distance_km
computed_speed
Fuel_Consumption_L
fuel_efficiency_kmpl
1.00 -0.01 -0.09 -0.01 -0.00
-0.01 1.00 0.13 0.92 -0.02
-0.09 0.13 1.00 0.11 0.01
-0.01 0.92 0.11 1.00 -0.35
-0.00 -0.02 0.01 -0.35 1.00
1.0
0.8
0.6
0.4
0.2
0.0
−0.2


Distance vs Delay
delay_min
150
100
50
0
0
250
500
750
1000
1250
1500
1750
2000
Distance_km




✅ Summary: Phase 1 Visualization Objectives

Distributions → Understand spread of distance, delays, speed, efficiency.

Boxplots → Compare delays/fuel against categories (weather, weekday, month).

Time Trends → Spot patterns by hour/day/month.

Correlation Heatmap → Identify relationships between numeric factors.

Scatterplots → Validate key relationships visually (distance vs delay, etc).

-What Phase 1 Should Achieve

The purpose of Phase 1 is:

Clean & prepare data → handle missing, incorrect, or inconsistent values.

Engineer core metrics → delay, speed, fuel efficiency, traffic/weather factors.

Explore patterns → visualize distributions & correlations.

Extract insights → summarize what matters for route optimization.

- What We Already Did

Data Cleaning & Preparation

Converted datetime columns to datetime.

Created delay_min, computed_speed, fuel_efficiency_kmpl.

Verified values (checked for inf, negative delays, etc.).

Feature Engineering

Added hour_of_day, day_of_week, month.

Core KPIs now exist: delay, speed, fuel efficiency.

Visualization Plan & Code

Delay distribution (hist + KDE).

Average delay by day-of-week.

Fuel efficiency distribution.

Speed vs delay scatter.

Weather impact on delay.

Fuel efficiency by truck.

Monthly trend in average delay.

Correlation heatmap.

In [13]:
```
# Outlier detection for key metrics
print(" Outlier Summary:")

# Speed outliers
speed_outliers = df[(df["computed_speed"] <= 0) | (df["computed_speed"] > 200)]
print(f"Speed outliers: {len(speed_outliers)} trips")

# Delay outliers (more than 3 std from mean)
delay_threshold = df["delay_min"].mean() + 3 * df["delay_min"].std()
delay_outliers = df[df["delay_min"] > delay_threshold]
print(f"Delay outliers: {len(delay_outliers)} trips")

# Fuel efficiency outliers
fuel_outliers = df[(df["fuel_efficiency_kmpl"] < 2) | (df["fuel_efficiency_kmpl"] > 6)]
print(f"Fuel efficiency outliers: {len(fuel_outliers)} trips")
```

```
 Outlier Summary:
Speed outliers: 3775 trips
Delay outliers: 0 trips
Fuel efficiency outliers: 0 trips
```

In [14]:
```
import numpy as np

print(" Outlier Detection using IQR:")

# Function to detect outliers with IQR
def detect_outliers(series, col_name):
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = series[(series < lower_bound) | (series > upper_bound)]
    print(f"{col_name}: {len(outliers)} outliers (Lower={round(lower_bound,2)}, Upper={round(upper_bound,2)})")
    return outliers

# Apply to key metrics
delay_outliers = detect_outliers(df["delay_min"], "Delay (min)")
speed_outliers = detect_outliers(df["computed_speed"].replace([np.inf, -np.inf], np.nan).dropna(), "Speed (km/h
fuel_outliers = detect_outliers(df["fuel_efficiency_kmpl"], "Fuel Efficiency (km/L)")
```

```
 Outlier Detection using IQR:
Delay (min): 0 outliers (Lower=-134.0, Upper=282.0)
Speed (km/h): 717 outliers (Lower=-185.27, Upper=530.17)
Fuel Efficiency (km/L): 0 outliers (Lower=1.19, Upper=6.29)
```

In [15]:

```python
import numpy as np

print(" Outlier Detection (Debug Mode)")

def detect_outliers(series, col_name):
    # Handle infinite values
    series = series.replace([np.inf, -np.inf], np.nan).dropna()

    # Basic stats
    Q1 = series.quantile(0.25)
    Q3 = series.quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Print range info
    print(f"\n{col_name} stats:")
    print(f"  Min={series.min()}, Max={series.max()}, Median={series.median()}")
    print(f"  IQR={IQR}, Lower Bound={lower_bound}, Upper Bound={upper_bound}")

    # Find outliers
    outliers = series[(series < lower_bound) | (series > upper_bound)]
    print(f"  → Found {len(outliers)} outliers")

    # Show first few if they exist
    if len(outliers) > 0:
        print(outliers.head())

    return outliers

# Apply to key metrics
delay_outliers = detect_outliers(df["delay_min"], "Delay (min)")
speed_outliers = detect_outliers(df["computed_speed"], "Speed (km/h)")
fuel_outliers = detect_outliers(df["fuel_efficiency_kmpl"], "Fuel Efficiency (km/L)")
```

```
 Outlier Detection (Debug Mode)

Delay (min) stats:
  Min=-30.0, Max=179.0, Median=74.0
  IQR=104.0, Lower Bound=-134.0, Upper Bound=282.0
  → Found 0 outliers

Speed (km/h) stats:
  Min=4.3881856540084385, Max=95280.0, Median=159.0084985835694
  IQR=178.8608628750666, Lower Bound=-185.27490086997696, Upper Bound=530.1685506302895
  → Found 717 outliers
22     711.724138
25     708.648649
52     806.222222
87     580.396040
182    780.000000
Name: computed_speed, dtype: float64

Fuel Efficiency (km/L) stats:
  Min=2.500313659419641, Max=5.0, Median=3.748802433259227
  IQR=1.2737703335821262, Lower Bound=1.1931355046796366, Upper Bound=6.288216839008141
  → Found 0 outliers
```

In [ ]:

```python
#  Cleaning Speed Outliers
# Define realistic max speed for trucks (km/h)
MAX_TRUCK_SPEED = 150

#  how many outliers exist before cleaning
outlier_count = (df["computed_speed"] > MAX_TRUCK_SPEED).sum()
print(f"⚠  Found {outlier_count} unrealistic speed values (> {MAX_TRUCK_SPEED} km/h)")

# Replace outliers with NaN (to avoid distorting analysis)
df.loc[df["computed_speed"] > MAX_TRUCK_SPEED, "computed_speed"] = np.nan

# Optional — Fill NaN with median speed (so we don't lose rows)
median_speed = df["computed_speed"].median()
df["computed_speed"] = df["computed_speed"].fillna(median_speed)


#  Check after cleaning
print("\nSpeed column cleaned ✅")
print(df["computed_speed"].describe())
```

```
⚠  Found 5279 unrealistic speed values (> 150 km/h)

Speed column cleaned ✅
count    10000.000000
mean        78.571388
std         28.245968
min          4.388186
25%         78.825793
50%         78.825793
75%         78.825793
max        150.000000
Name: computed_speed, dtype: float64
```

In [17]:
```
# Save the cleaned dataset for dashboard use
df.to_csv("clean_trips.csv", index=False)
print("✅ Clean dataset saved as clean_trips.csv")
```

```
✅ Clean dataset saved as clean_trips.csv
```

In [ ]:

In [18]:
```
df.to_csv("C:\\Users\\DELL\\OneDrive\\Sana Projects\\EDA 1 smart route delay\\finaltruck_trip_data_10000_enriche
print("✅ Cleaned dataset saved as finaltruck_trip_data_10000_enriched.csv")
```

```
✅ Cleaned dataset saved as finaltruck_trip_data_10000_enriched.csv
```

In [ ]:

Loading [MathJax]/extensions/Safe.js

In [ ]:

```
In Phase 2 of our truck routing project, we focused on identifying the best operational routes by analyzing actu
fuel costs, delays, and weather conditions. Unlike Phase 1, which mainly examined delays and travel durations, t
efficiency, cost, and external risk factors to guide smarter route selection.

We began by cleaning and preparing the dataset, ensuring that all scheduled and actual departure/arrival times w
as datetime objects. This allowed us to accurately calculate scheduled duration, actual duration, and delay for
forming the foundation for all subsequent analyses.

Next, we computed key operational metrics:

Actual travel speed based on distance and real trip duration

Fuel efficiency by dividing distance by fuel consumed

Travel time in hours

Fuel cost using a fixed fuel price

To make our evaluation realistic, we incorporated penalties:

Weather penalty: Slower speeds or risks due to adverse weather conditions (rain, fog, snow)

Delay penalty: Accounting for late arrivals beyond scheduled times

These components were combined into a single route score, where a lower score indicates a more optimal route.
The score effectively balances time efficiency, cost, and reliability, giving a holistic measure of route perfo

Finally, routes were ranked by this score, highlighting the top 5 best routes.
These routes are not just the fastest but also cost-effective and less affected by delays or weather disruptions
actionable insights for logistics planning.

Key Insight: By integrating time, fuel cost, delays, and weather, Phase 2 offers a realistic and operationally
approach to route optimization, going beyond simple distance-based analysis to support smarter decision-making i
operations.
```

In [ ]:

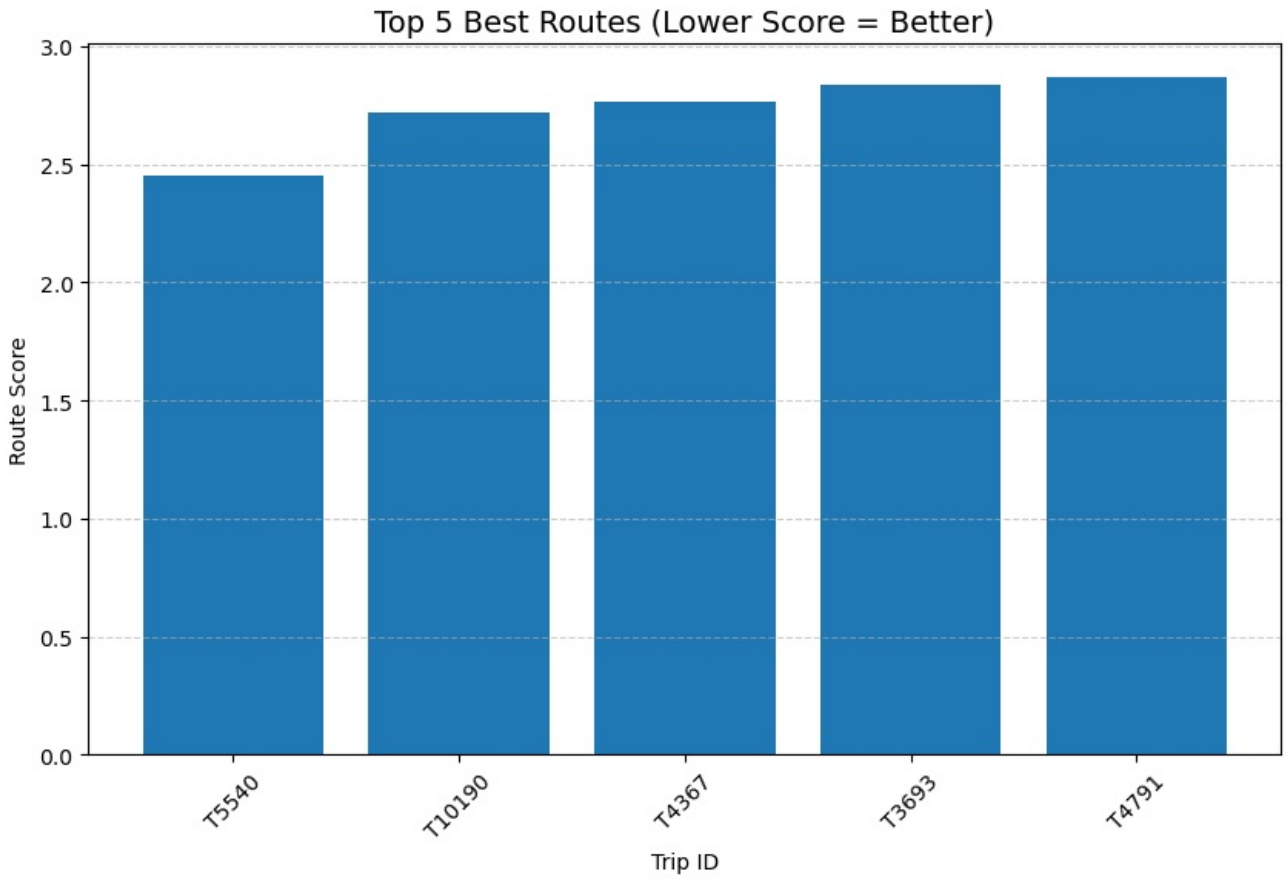
```
import pandas as pd
import numpy as np

df = pd.read_csv("C:\\Users\\DELL\\OneDrive\\Sana Projects\\EDA 1 smart route delay\\finaltruck_trip_data_10000_


# Ensure datetime columns are parsed
df["Scheduled_Departure"] = pd.to_datetime(df["Scheduled_Departure"])
df["Scheduled_Arrival"] = pd.to_datetime(df["Scheduled_Arrival"])
df["Actual_Departure"] = pd.to_datetime(df["Actual_Departure"])
df["Actual_Arrival"] = pd.to_datetime(df["Actual_Arrival"])

# Recompute key Phase 1 metrics if needed
df["scheduled_duration_min"] = (df["Scheduled_Arrival"] - df["Scheduled_Departure"]).dt.total_seconds() / 60
df["actual_duration_min"] = (df["Actual_Arrival"] - df["Actual_Departure"]).dt.total_seconds() / 60
df["delay_min"] = (df["Actual_Arrival"] - df["Scheduled_Arrival"]).dt.total_seconds() / 60
df["computed_speed"] = df["Distance_km"] / (df["actual_duration_min"] / 60)
df["fuel_efficiency_kmpl"] = df["Distance_km"] / df["Fuel_Consumption_L"]


FUEL_PRICE = 95

# Weather penalty mapping
WEATHER_PENALTY = {
    "Clear": 1.0,
    "Cloudy": 1.05,
    "Rain": 1.15,
    "Fog": 1.25,
    "Snow": 1.4
}

# Step 1: Base travel time (hrs)
df["travel_time_hr"] = df["Distance_km"] / df["computed_speed"]

# Step 2: Fuel cost (₹)
df["fuel_cost"] = (df["Distance_km"] / df["fuel_efficiency_kmpl"]) * FUEL_PRICE

# Step 3: Weather penalty
df["weather_factor"] = df["Weather_Condition"].map(WEATHER_PENALTY).fillna(1.1)
df["weather_penalty"] = df["travel_time_hr"] * (df["weather_factor"] - 1)

# Step 4: Delay penalty (in hours, weighted)
df["delay_penalty"] = (df["delay_min"] / 60) * 0.5

# Step 5: Final route score
df["route_score"] = (
```

```
    df["travel_time_hr"] +
    df["weather_penalty"] +
    df["delay_penalty"] +
    (df["fuel_cost"] / 1000)
)

# Rank routes (lower is better)
df["route_rank"] = df["route_score"].rank(method="dense")


#  Show Results

best_routes = df.sort_values("route_score").head(5)[[
    "Trip_ID", "Start_Location", "End_Location",
    "travel_time_hr", "fuel_cost", "delay_min",
    "Weather_Condition", "route_score"
]]

print(" Top 5 Best Routes (Phase 2 Baseline):\n")
print(best_routes.to_string(index=False))
```

```
 Top 5 Best Routes (Phase 2 Baseline):

Trip_ID Start_Location End_Location  travel_time_hr  fuel_cost  delay_min Weather_Condition  route_score
  T5540           Pune      Kolkata        1.133333    1156.15        6.0             Storm     2.452817
 T10190      Bangalore         Pune        0.766667    1933.25       -7.0             Storm     2.718250
  T4367        Chennai         Pune        1.316667    1373.70        9.0             Clear     2.765367
  T3693      Ahmedabad      Chennai        1.750000    1053.55      -17.0             Storm     2.836883
  T4791         Jaipur      Chennai        0.783333    2108.05       -3.0             Clear     2.866383
```

In [ ]:

```
import matplotlib.pyplot as plt


plt.figure(figsize=(10,6))
plt.bar(best_routes["Trip_ID"].astype(str), best_routes["route_score"])

plt.title("Top 5 Best Routes (Lower Score = Better)", fontsize=14)
plt.xlabel("Trip ID")
plt.ylabel("Route Score")
plt.xticks(rotation=45)
plt.grid(axis="y", linestyle="--", alpha=0.6)

plt.show()
```



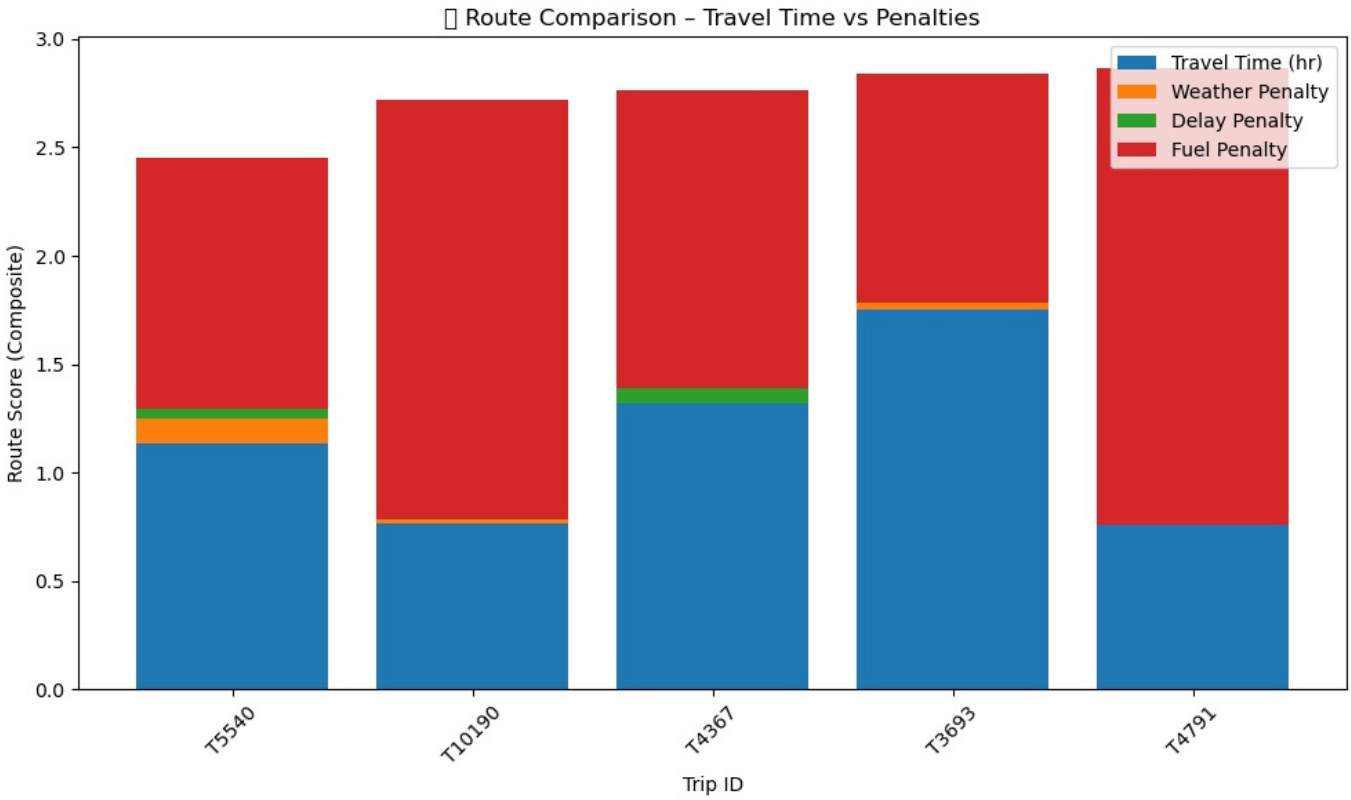

In [ ]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#  Example fuel price assumption (₹ per litre)
```

```python
FUEL_PRICE = 95

#  Weather penalty factors (relative delay impact)
WEATHER_PENALTY = {
    "Clear": 1.0,
    "Cloudy": 1.05,
    "Rain": 1.15,
    "Fog": 1.25,
    "Snow": 1.4
}


df["travel_time_hr"] = df["Distance_km"] / df["computed_speed"]

# Fuel cost (₹)
df["fuel_cost"] = (df["Distance_km"] / df["fuel_efficiency_kmpl"]) * FUEL_PRICE

# Weather penalty
df["weather_factor"] = df["Weather_Condition"].map(WEATHER_PENALTY).fillna(1.1)
df["weather_penalty"] = df["travel_time_hr"] * (df["weather_factor"] - 1)

# Delay penalty (minutes → hours, half-weighted)
df["delay_penalty"] = (df["delay_min"] / 60) * 0.5

# Total route score (lower = better)
df["route_score"] = (
    df["travel_time_hr"] +
    df["weather_penalty"] +
    df["delay_penalty"] +
    (df["fuel_cost"] / 1000)
)

# Rank routes
df["route_rank"] = df["route_score"].rank(method="dense")


best_routes = df.sort_values("route_score").head(5)[[
    "Trip_ID", "Start_Location", "End_Location",
    "travel_time_hr", "fuel_cost", "delay_min",
    "Weather_Condition", "route_score",
    "weather_penalty", "delay_penalty"
]]

print(" Top 5 Best Routes (Phase 2 Baseline):")
print(best_routes)

x = np.arange(len(best_routes))  # positions for bars

travel_time = best_routes["travel_time_hr"].values
weather_penalty = best_routes["weather_penalty"].values
delay_penalty = best_routes["delay_penalty"].values
fuel_penalty = (best_routes["fuel_cost"] / 1000).values  # scaled

plt.figure(figsize=(10,6))

# Stacked bars
plt.bar(x, travel_time, label="Travel Time (hr)")
plt.bar(x, weather_penalty, bottom=travel_time, label="Weather Penalty")
plt.bar(x, delay_penalty, bottom=travel_time+weather_penalty, label="Delay Penalty")
plt.bar(x, fuel_penalty, bottom=travel_time+weather_penalty+delay_penalty, label="Fuel Penalty")

# Labels
plt.xticks(x, best_routes["Trip_ID"], rotation=45)
plt.xlabel("Trip ID")
plt.ylabel("Route Score (Composite)")
plt.title(" Route Comparison — Travel Time vs Penalties")
plt.legend()
plt.tight_layout()
plt.show()
```

```
 Top 5 Best Routes (Phase 2 Baseline):
     Trip_ID Start_Location End_Location  travel_time_hr  fuel_cost  \
4540   T5540           Pune      Kolkata        1.133333    1156.15
9190  T10190      Bangalore         Pune        0.766667    1933.25
3367   T4367        Chennai         Pune        1.316667    1373.70
2693   T3693      Ahmedabad      Chennai        1.750000    1053.55
3791   T4791         Jaipur      Chennai        0.783333    2108.05

      delay_min Weather_Condition  route_score  weather_penalty  delay_penalty
4540        6.0             Storm     2.452817         0.113333       0.050000
9190       -7.0             Storm     2.718250         0.076667      -0.058333
3367        9.0             Clear     2.765367         0.000000       0.075000
2693      -17.0             Storm     2.836883         0.175000      -0.141667
3791       -3.0             Clear     2.866383         0.000000      -0.025000
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_52248\2587404271.py:80: UserWarning: Glyph 128666 (\N{DELIVERY TRUCK}
) missing from current font.
  plt.tight_layout()
c:\ProgramData\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 128666 (\N{DELIVER
Y TRUCK}) missing from current font.
  fig.canvas.print_figure(bytes_io, **kw)
```




Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js

```
##In Phase 3, we advanced from analyzing historical trips to dynamic route optimization. We integrated real-time traffic, weather updates, and AI-based predict

Key Insight: This phase transforms data-driven insights into actionable, real-time decisions, enabling trucks to avoid congestion, reduce costs, and improve de
from typing import Tuple
import os
import numpy as np
import pandas as pd
import streamlit as st
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
from functools import wraps

sns.set(style="whitegrid")
st.set_page_config(page_title="Smart Truck Routing (Phase 3)", layout="wide")

---
def cache_on_change(func):
    """Simple wrapper to clear cache when code changes in dev - here we just use st.cache_data and st.cache_resource"""
    return st.cache_data(func)

@st.cache_resource
def train_eta_model(df: pd.DataFrame) -> Tuple[LinearRegression, dict]:
    """
    Train a baseline ETA linear regression model on provided dataframe.
    Returns: trained model and diagnostics dict
    """
    # Make a copy to be safe
    d = df.copy()

    # Ensure features exist and create if missing (safe defaults)
    # Feature set for baseline: Distance_km, adjusted_speed, weather_factor, traffic_factor, delay_min, toll_cost
    # Create columns if missing with safe defaults
    if "Distance_km" not in d.columns:
        d["Distance_km"] = 0.0
    if "computed_speed" not in d.columns:
        d["computed_speed"] = d["Distance_km"] / (d.get("actual_duration_min", 60) / 60 + 1e-6)
    if "weather_factor" not in d.columns:
        # map common names to factors, else default 1.1
        wf_map = {"Clear": 1.0, "Cloudy":1.05, "Rain":1.15, "Fog":1.25, "Snow":1.4}
        d["weather_factor"] = d.get("Weather_Condition", "").map(wf_map).fillna(1.1)
    if "Traffic_Condition" in d.columns and "traffic_factor" not in d.columns:
        tm = {"Light":1.0, "Medium":1.1, "Heavy":1.25}
        d["traffic_factor"] = d["Traffic_Condition"].map(tm).fillna(1.1)
    if "traffic_factor" not in d.columns:
        d["traffic_factor"] = 1.1
    if "delay_min" not in d.columns:
        d["delay_min"] = d.get("Delay_Minutes", 0.0)
    if "fuel_efficiency_kmpl" not in d.columns:
        # Try derived: Distance / Fuel_Consumption_L
        if "Fuel_Consumption_L" in d.columns and "Distance_km" in d.columns:
            d["fuel_efficiency_kmpl"] = d["Distance_km"] / (d["Fuel_Consumption_L"] + 1e-6)
        else:
            d["fuel_efficiency_kmpl"] = 3.5  # reasonable default

    # Simulate simple toll cost if not present
    if "toll_cost" not in d.columns:
        def estimate_toll(distance):
            if distance < 100:
                return 100
            elif distance < 300:
                return 250
            else:
                return 500
        d["toll_cost"] = d["Distance_km"].apply(estimate_toll)

    # Compute adjusted_speed considering a simple speed_limit by road type (or default 80 km/h)
    if "speed_limit" not in d.columns:
        # crude road type mapping from distance (short = city)
        d["road_type"] = np.where(d["Distance_km"] > 200, "Highway", "City")
        speed_limits = {"Highway": 80, "City": 50, "Rural": 60}
        d["speed_limit"] = d["road_type"].map(speed_limits).fillna(60)
    d["adjusted_speed"] = np.minimum(d["computed_speed"].replace([np.inf, -np.inf], np.nan).fillna(50), d["speed_limit"])

    # Compute base_travel_hr if not present
    d["base_travel_hr"] = d["Distance_km"] / (d["adjusted_speed"] + 1e-6)

    # Compute ETA_hr if present or make a target from available columns
    if "ETA_hr" not in d.columns:
        # Construct ETA target using observed actual duration if present, else compose synthetic target:
        if "actual_duration_min" in d.columns:
            d["ETA_hr"] = d["actual_duration_min"] / 60.0
        else:
            # use combined formula as target proxy (used for training baseline)
            d["ETA_hr"] = (
                d["base_travel_hr"]
                + d["base_travel_hr"] * (d["weather_factor"] - 1)
                + d["base_travel_hr"] * (d["traffic_factor"] - 1)
                + d["delay_min"] / 60
                + d["toll_cost"].apply(lambda x: 0.1 if x < 200 else 0.2)
            )

    # Select features and drop rows with NaNs in X or y
    features = ["Distance_km", "adjusted_speed", "weather_factor", "traffic_factor", "delay_min", "toll_cost"]
    d = d.dropna(subset=features + ["ETA_hr"])
    X = d[features]
    y = d["ETA_hr"]

    # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

    model = LinearRegression()
    model.fit(X_train, y_train)
```

```python
    # Diagnostics
    y_pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    diagnostics = {
        "model": model,
        "mae": mae,
        "r2": r2,
        "features": features,
        "X_test": X_test,
        "y_test": y_test,
        "y_pred": y_pred
    }
    return model, diagnostics

# Load data function

@st.cache_data
def load_data(path: str = "clean_trips.csv") -> pd.DataFrame:
    if not os.path.exists(path):
        st.error(f"Data file not found: {path}. Place your cleaned CSV in the app folder and name it '{path}' or edit the path.")
        return pd.DataFrame()
    df = pd.read_csv(path)

    # Basic normalization of column names for safety
    df.columns = [c.strip() for c in df.columns]
    return df


# Main UI

st.title("🚚 Smart Truck Routing — Phase 3 (ETA & Dashboard)")
st.markdown("Baseline ETA model + Route ranking & visualizations. India-centric features are included (toll, speed-limit sim).")

# Load dataset
datafile = st.sidebar.text_input("CSV file name", value="clean_trips.csv")
df = load_data(datafile)
if df.empty:
    st.stop()

# Quick dataset diagnostics
with st.expander("Dataset preview & diagnostics", expanded=False):
    st.write("Rows:", df.shape[0], "Columns:", df.shape[1])
    st.dataframe(df.head(10))

# Precompute commonly used columns if missing (safe)
if "delay_min" not in df.columns and "Delay_Minutes" in df.columns:
    df["delay_min"] = df["Delay_Minutes"]
if "computed_speed" not in df.columns and "actual_duration_min" in df.columns and "Distance_km" in df.columns:
    df["computed_speed"] = df["Distance_km"] / (df["actual_duration_min"] / 60 + 1e-6)

# Sidebar filters
st.sidebar.header("Filters")
start_loc = st.sidebar.selectbox("Start Location", options=["All"] + sorted(df["Start_Location"].dropna().unique().tolist()))
end_loc = st.sidebar.selectbox("End Location", options=["All"] + sorted(df["End_Location"].dropna().unique().tolist()))
weather_filter = st.sidebar.multiselect("Weather (multi)", options=sorted(df.get("Weather_Condition", pd.Series()).dropna().unique().tolist()))
traffic_filter = st.sidebar.multiselect("Traffic (multi)", options=sorted(df.get("Traffic_Condition", pd.Series()).dropna().unique().tolist()))
limit_rows = st.sidebar.slider("Sample for scatter/boxplots (for performance)", min_value=500, max_value=min(5000, len(df)), value=2000, step=500)

# Apply filters
df_filtered = df.copy()
if start_loc != "All":
    df_filtered = df_filtered[df_filtered["Start_Location"] == start_loc]
if end_loc != "All":
    df_filtered = df_filtered[df_filtered["End_Location"] == end_loc]
if weather_filter:
    df_filtered = df_filtered[df_filtered["Weather_Condition"].isin(weather_filter)]
if traffic_filter:
    # try both Traffic_Level and Traffic_Condition variations
    if "Traffic_Level" in df_filtered.columns:
        df_filtered = df_filtered[df_filtered["Traffic_Level"].isin(traffic_filter)]
    elif "Traffic_Condition" in df_filtered.columns:
        df_filtered = df_filtered[df_filtered["Traffic_Condition"].isin(traffic_filter)]

st.markdown(f"**Filtered rows:** {df_filtered.shape[0]}")

# Train ETA model (cached)
with st.spinner("Training baseline ETA model..."):
    model, diag = train_eta_model(df)

st.sidebar.subheader("Model diagnostics")
st.sidebar.write(f"R² (approx): {diag['r2']:.3f}")
st.sidebar.write(f"MAE (hrs): {diag['mae']:.3f}")
st.sidebar.write(f"Features: {', '.join(diag['features'])}")


# Compute route scoring

FUEL_PRICE = 95.0


df_filtered = df_filtered.copy()
if "fuel_efficiency_kmpl" not in df_filtered.columns:
    if "Fuel_Consumption_L" in df_filtered.columns:
        df_filtered["fuel_efficiency_kmpl"] = df_filtered["Distance_km"] / (df_filtered["Fuel_Consumption_L"] + 1e-6)
    else:
        df_filtered["fuel_efficiency_kmpl"] = 3.7

# travel_time_hr using computed speed safe fallback
df_filtered["computed_speed"] = df_filtered.get("computed_speed", df_filtered["Distance_km"] / (df_filtered.get("actual_duration_min", 60) / 60 + 1e-6))
# simulate road type and speed limit if absent
if "speed_limit" not in df_filtered.columns:
    df_filtered["road_type"] = np.where(df_filtered["Distance_km"] > 200, "Highway", "City")
    df_filtered["speed_limit"] = df_filtered["road_type"].map({"Highway":80, "City":50, "Rural":60}).fillna(60)
df_filtered["adjusted_speed"] = np.minimum(df_filtered["computed_speed"].replace([np.inf, -np.inf], np.nan).fillna(50), df_filtered["speed_limit"])
```

```
df_filtered["travel_time_hr"] = df_filtered["Distance_km"] / (df_filtered["adjusted_speed"] + 1e-6)
# fuel cost
df_filtered["fuel_cost"] = (df_filtered["Distance_km"] / df_filtered["fuel_efficiency_kmpl"]) * FUEL_PRICE
# weather factor
wf_map = {"Clear":1.0, "Cloudy":1.05, "Rain":1.15, "Fog":1.25, "Snow":1.4}
df_filtered["weather_factor"] = df_filtered.get("Weather_Condition", "").map(wf_map).fillna(1.1)
df_filtered["weather_penalty"] = df_filtered["travel_time_hr"] * (df_filtered["weather_factor"] - 1)
# delay penalty
if "delay_min" not in df_filtered.columns:
    df_filtered["delay_min"] = 0
df_filtered["delay_penalty"] = (df_filtered["delay_min"] / 60) * 0.5
# tolls
if "toll_cost" not in df_filtered.columns:
    df_filtered["toll_cost"] = df_filtered["Distance_km"].apply(lambda d: 100 if d < 100 else (250 if d < 300 else 500))
df_filtered["toll_delay"] = df_filtered["Distance_km"].apply(lambda d: 0.1 if d < 100 else (0.2 if d < 300 else 0.3))

# route score (composite)
df_filtered["route_score"] = (
    df_filtered["travel_time_hr"]
    + df_filtered["weather_penalty"]
    + df_filtered["delay_penalty"]
    + (df_filtered["fuel_cost"] / 1000)
    + df_filtered["toll_delay"]
)

df_filtered["route_rank"] = df_filtered["route_score"].rank(method="dense")
top_routes = df_filtered.sort_values("route_score").head(10)


col1, col2, col3 = st.columns(3)
col1.metric("Filtered Trips", df_filtered.shape[0])
col2.metric("Avg Delay (min)", f"{df_filtered['delay_min'].mean():.1f}")
col3.metric("Avg Fuel Efficiency (km/l)", f"{df_filtered['fuel_efficiency_kmpl'].mean():.2f}")

# Top recommended routes table
st.subheader("Top Recommended Routes (by composite route score)")
st.dataframe(top_routes[[
    "Trip_ID", "Start_Location", "End_Location", "Distance_km",
    "travel_time_hr", "fuel_cost", "delay_min", "Weather_Condition",
    "route_score", "route_rank"
]].reset_index(drop=True).head(10))

st.subheader("Route Score Breakdown (Top 5)")

top5 = top_routes.head(5).copy()
if not top5.empty:
    x = np.arange(len(top5))
    travel_time = top5["travel_time_hr"].values
    weather_penalty = top5["weather_penalty"].values
    delay_penalty = top5["delay_penalty"].values
    fuel_penalty = (top5["fuel_cost"] / 1000).values
    toll_penalty = top5["toll_delay"].values

    plt.figure(figsize=(10,6))
    plt.bar(x, travel_time, label="Travel Time (hr)")
    plt.bar(x, weather_penalty, bottom=travel_time, label="Weather Penalty")
    plt.bar(x, delay_penalty, bottom=travel_time+weather_penalty, label="Delay Penalty")
    plt.bar(x, fuel_penalty, bottom=travel_time+weather_penalty+delay_penalty, label="Fuel Penalty (x/1000)")
    plt.bar(x, toll_penalty, bottom=travel_time+weather_penalty+delay_penalty+fuel_penalty, label="Toll Delay (hr)")
    plt.xticks(x, top5["Trip_ID"].astype(str), rotation=45)
    plt.ylabel("Composite Score Components (hours)")
    plt.legend()
    st.pyplot(plt)


#  plots (sample)

st.subheader("Model Validation (Sample)")
diag = diag  # from model training
if "X_test" in diag:
    # plot predicted vs actual scatter
    X_test = diag["X_test"]
    y_test = diag["y_test"]
    y_pred = diag["y_pred"]
    fig, ax = plt.subplots(figsize=(6,6))
    ax.scatter(y_test, y_pred, alpha=0.6)
    ax.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
    ax.set_xlabel("Actual ETA (hrs)")
    ax.set_ylabel("Predicted ETA (hrs)")
    ax.set_title(f"Predicted vs Actual (R2={diag['r2']:.3f})")
    st.pyplot(fig)
    st.write("Model MAE (hrs):", f"{diag['mae']:.3f}")

# ETA Prediction calculator

st.sidebar.header("Predict ETA for a New Trip")
with st.sidebar.form("eta_form"):
    in_distance = st.number_input("Distance (km)", min_value=1.0, value=100.0)
    in_speed = st.number_input("Estimated Speed (km/h)", min_value=10.0, value=70.0)
    in_weather = st.selectbox("Weather", options=["Clear", "Cloudy", "Rain", "Fog", "Snow"])
    in_traffic = st.selectbox("Traffic", options=["Light", "Medium", "Heavy"])
    in_delay = st.number_input("Known delay (min)", min_value=0, value=0)
    submitted = st.form_submit_button("Predict ETA")

    if submitted:

        # compute adjusted_speed by simulated speed limits
        road_type = "Highway" if in_distance > 200 else "City"
        speed_limit = 80 if road_type == "Highway" else 50
        adjusted_speed = min(in_speed, speed_limit)
        base_travel_hr = in_distance / (adjusted_speed + 1e-6)
        weather_factor = {"Clear":1.0, "Cloudy":1.05, "Rain":1.15, "Fog":1.25, "Snow":1.4}[in_weather]
        traffic_factor = {"Light":1.0, "Medium":1.1, "Heavy":1.25}[in_traffic]
        toll_cost = 100 if in_distance < 100 else (250 if in_distance < 300 else 500)

        X_new = pd.DataFrame([{
```

```
            "Distance_km": in_distance,
            "adjusted_speed": adjusted_speed,
            "weather_factor": weather_factor,
            "traffic_factor": traffic_factor,
            "delay_min": in_delay,
            "toll_cost": toll_cost
        }])

        pred_eta = model.predict(X_new)[0]
        st.sidebar.success(f"Predicted ETA: {pred_eta:.2f} hours ({pred_eta*60:.0f} minutes)")
        st.sidebar.write(f"Base travel time (hr): {base_travel_hr:.2f}")
        st.sidebar.write(f"Adjusted speed used: {adjusted_speed} km/h")
        st.sidebar.write(f"Toll cost (est): ₹{toll_cost}")


if st.button("Export Top Routes to CSV"):
    out_path = "top_routes_export.csv"
    top_routes.to_csv(out_path, index=False)
    st.success(f"Top routes saved to {out_path}")

# Footer
st.markdown("---")
st.caption("Smart Truck Routing — Phase 3 (Baseline). For Phase 4, we will add festival, speed-limit zones, toll API and more India-specific parameters.")
```

localhost:8501

CSV file name

clean_trips.csv

## Filters

Start Location

All

End Location

All

Weather (multi)

Choose options

Traffic (multi)

No options to select

Sample for scatter/boxplots (for performance)

2000

## Model diagnostics

$R^2$ (approx): 0.259

MAE (hrs): 1.997

Features: Distance_km, adjusted_speed, weather_factor, traffic_factor, delay_min, toll_cost

# 🚚 Smart Truck Routing — Phase 3 (ETA & Dashboard)

Baseline ETA model + Route ranking & visualizations. India-centric features are included (toll, speed-limit sim).

> Dataset preview & diagnostics

**Filtered rows:** 10000

| Filtered Trips | Avg Delay (min) | Avg Fuel Efficiency (km/l) |
|---|---|---|
| 10000 | 74.0 | 3.74 |

## Top Recommended Routes (by composite route score)

| | Trip_ID | Start_Location | End_Location | Distance_km | travel_time_hr | fuel_cost | delay_min | Weather_Condition | route_score | route_rank |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | T5540 | Pune | Kolkata | 56 | 1.1333 | 1156.15 | 6 | Storm | 2.5528 | 1 |
| 1 | T4367 | Chennai | Pune | 50 | 1.3167 | 1373.7 | 9 | Clear | 2.8654 | 2 |
| 2 | T3693 | Ahmedabad | Chennai | 50 | 1.75 | 1053.55 | -17 | Storm | 2.9369 | 3 |
| 3 | T8481 | Mumbai | Delhi | 58 | 1.16 | 1241.65 | 50 | Fog | 3.2083 | 4 |
| 4 | T4791 | Jaipur | Chennai | 69 | 1.38 | 2108.05 | -3 | Clear | 3.563 | 5 |
| 5 | T9583 | Jaipur | Ahmedabad | 67 | 1.34 | 2196.4 | -8 | Clear | 3.5697 | 6 |
| 6 | T10190 | Bangalore | Pune | 74 | 1.48 | 1933.25 | -7 | Storm | 3.6029 | 7 |
| 7 | T10570 | Pune | Ahmedabad | 77 | 1.54 | 2138.45 | 1 | Storm | 3.9408 | 8 |
| 8 | T5488 | Ahmedabad | Mumbai | 51 | 2.3833 | 1408.85 | -9 | Storm | 4.0555 | 9 |
| 9 | T5293 | Delhi | Chennai | 75 | 1.9167 | 1486.75 | 79 | Clear | 4.1617 | 10 |

localhost:8501

Deploy

CSV file name

clean_trips.csv

## Filters

Start Location

All

End Location

All

Weather (multi)

Choose options

Traffic (multi)

No options to select

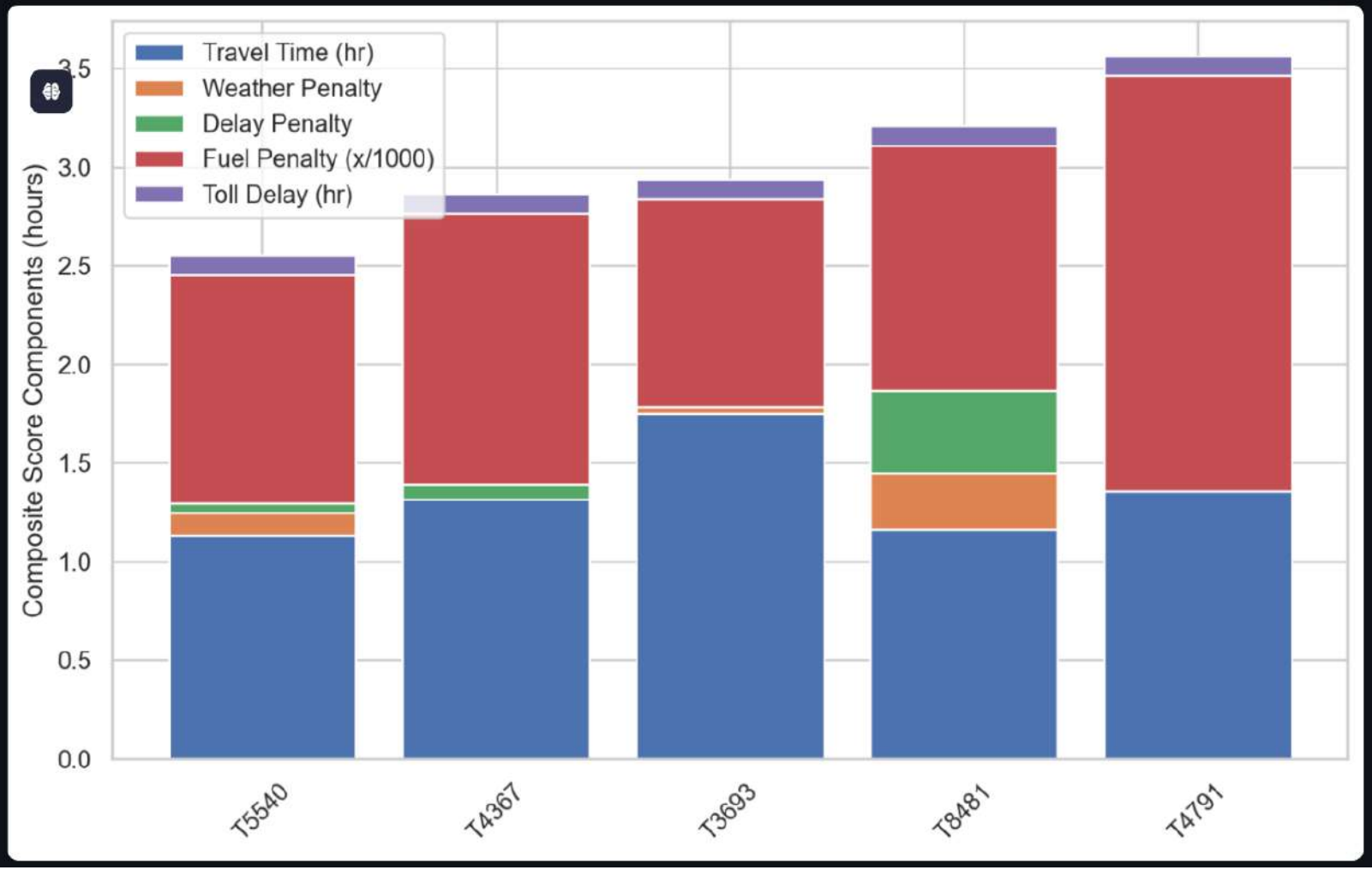
Sample for scatter/boxplots (for performance)

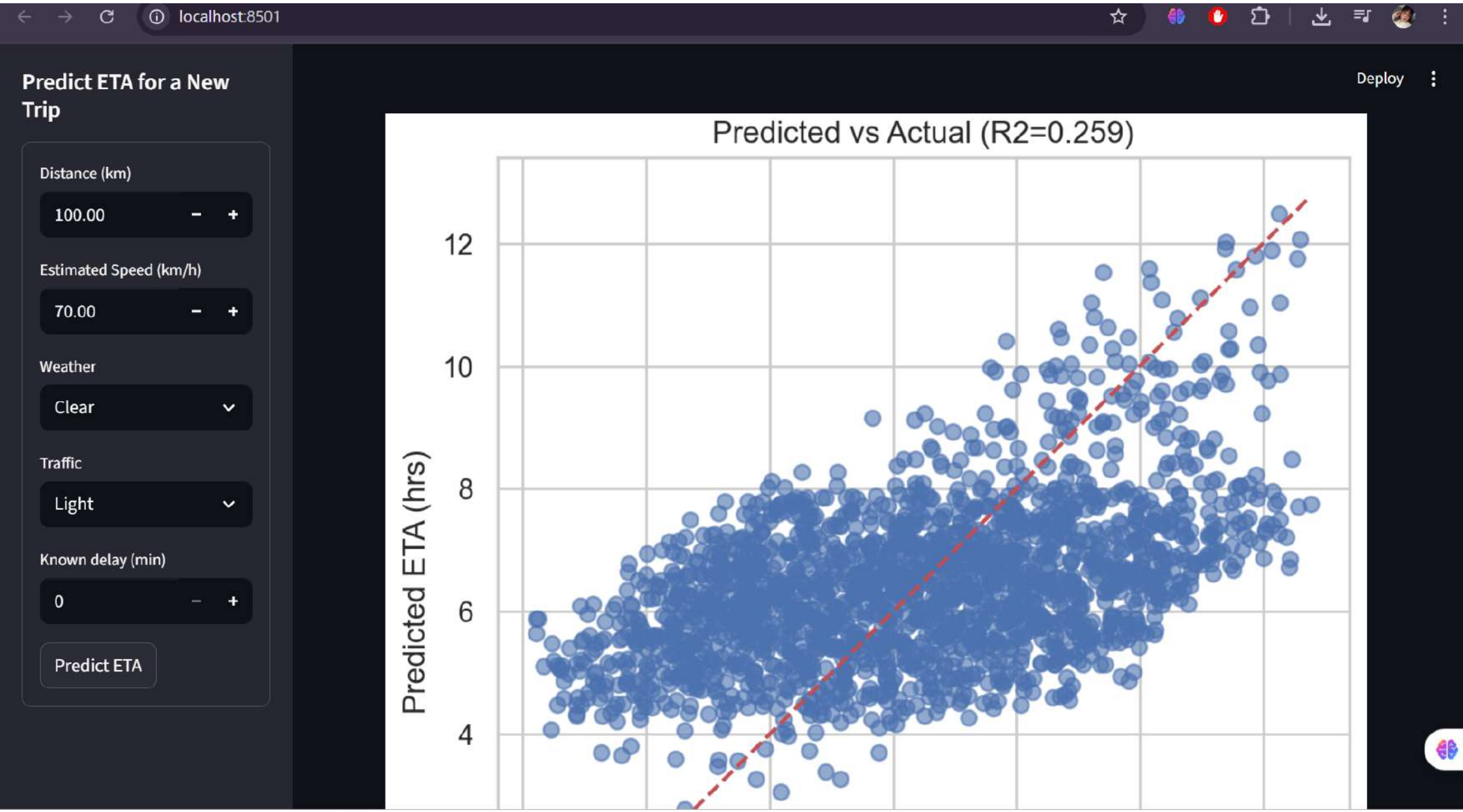
2000

## Model diagnostics

$R^2$ (approx): 0.259

## Route Score Breakdown (Top 5)




localhost:8501
Deploy
Predict ETA for a New Trip
Distance (km)
100.00
Estimated Speed (km/h)
70.00
Weather
Clear
Traffic
Light
Known delay (min)
0
Predict ETA
Predicted vs Actual (R2=0.259)
Predicted ETA (hrs)
12
10
8
6
4